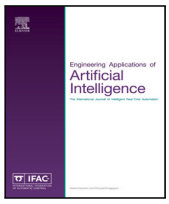
Engineering Applications of Artificial Intelligence 171 (2026) 114284



Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai





A trend-aware reinforcement learning approach for adaptive motion planning of robotic manipulators in dynamic environments

Dexian Wang [a,b,c], Peng Zhang [b,c,*], Pengfei Ding [b,c,d], Junliang Wang [b,c], Jie Zhang [b,c]

[a] College of Information Science and Technology, Donghua University, Shanghai, 201620, China
[b] Engineering Research Center of Artificial Intelligence for Textile Industry Ministry of Education, Institute of Artificial Intelligence, Donghua University, Shanghai, 201620, China
[c] Shanghai Engineering Research Center of Industrial Big Data and Intelligent System, Institute of Artificial Intelligence, Donghua University, Shanghai, 201620, China
[d] College of Mechanical Engineering, Donghua University, Shanghai, 201620, China

ARTICLE INFO

Keywords:
Reinforcement learning
Dynamic environments
Robotic manipulators motion planning
Trend learning
Reward shaping
Distillation

ABSTRACT

In engineering applications such as service robotics, human–robot collaboration, and industrial automation, robotic manipulators frequently operate in highly dynamic and partially observable environments. The presence of dynamic obstacles, unpredictable human behaviors, and rapidly changing task demands intensify the inherent conflicts between positioning accuracy, obstacle avoidance, and operational safety. At the same time, these systems must rely on incomplete and noisy sensor data to perceive and interpret their surroundings. Such dynamic scenarios significantly increase system complexity and require adaptive, real-time control strategies capable of making reliable decisions under uncertainty. To address these challenges, this paper proposes Trend Learning – Adaptive Reward Shaping – Temporal Difference Knowledge Distillation of Q-value(Action-Value Function), collectively referred to as TL-ARS-TDKDQ, a reinforcement learning framework designed to enable robotic manipulators to adaptively perform precise positioning and dynamic obstacle avoidance in dynamic environments. Trend Learning(TL) alleviates environmental uncertainty by extracting temporal dependencies from sequential data of the manipulator. Adaptive Reward Shaping(ARS) dynamically balances positioning accuracy and obstacle avoidance for the robotic manipulator while progressively increasing task difficulty via curriculum learning. To enhance stability during reward fluctuations caused by ARS,Temporal Difference Knowledge Distillation Q-value (TDKDQ) employs a dynamic teacher network and Temporal Difference(TD) error-based balancing, ensuring stable policy convergence in non-stationary scenarios involving robotic manipulator control. Experiments with a KUKA arm in CoppeliaSim demonstrate that TL-ARS-TDKDQ significantly improves convergence speed, control stability, and task success when integrated into mainstream continuous control reinforcement learning algorithms.

1. Introduction

Robotic manipulators are extensively employed in manufacturing, medical, and assembly applications due to their flexibility and ease of control (Ortega-Aranda et al., 2021; Rao et al., 2017; Szczepanski et al., 2022). Their operating workspaces often include both static and moving obstacles, resulting in constrained rather than open environments. Furthermore, these environments are typically dynamic and uncertain, characterized by unpredictable changes and incomplete information. Under such conditions, manipulators must perform real-time collision avoidance while maintaining precise control over the position and orientation of their end-effectors. Therefore, developing efficient path planning methods that can resolve conflicting requirements — such as positioning accuracy, obstacle avoidance, and operational safety — in dynamic, constrained, and uncertain environments remains a significant and practically valuable research challenge.

In recent years, path planning for robotic manipulators operating in dynamic and constrained environments has garnered significant research interest. This is largely motivated by the escalating complexity of practical applications, which require the development of reliable and effective algorithms to navigate safely around both stationary and

Abbreviations: TL-ARS-TDKDQ, Trend Learning – Adaptive Reward Shaping – Temporal Difference Knowledge Distillation Q-value

* Correspondence to: Donghua University, Room 2408, College Building, No. 2, 2999 Renmin North Road, Songjiang District, Shanghai, China.
E-mail addresses: 2232212@mail.dhu.edu.cn (D. Wang), zhangp88@dhu.edu.cn (P. Zhang), pengfeiding@mail.dhu.edu.cn (P. Ding), junliangwang@dhu.edu.cn (J. Wang), mezhangjie@dhu.edu.cn (J. Zhang).

https://doi.org/10.1016/j.engappai.2026.114284
Received 8 July 2025; Received in revised form 20 January 2026; Accepted 20 February 2026
Available online 23 February 2026
0952-1976/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

moving obstacles, while achieving accurate control of the manipulator's end-effector path. Traditional approaches, such as the artificial potential field (APF) method, have been extensively explored. Lin et al. proposed a three-dimensional path planning strategy based on APF, effectively mitigating the local minima problem and enabling manipulators to safely pass through narrow passages (Lin et al., 2025). Meanwhile, graph search algorithms including A* and Dijkstra have also been applied in manipulator trajectory planning. An enhanced D* algorithm was presented by Chen et al. which optimized pre-planned global paths by reducing turning points and minimizing collision risk, while a real-time path adjustment mechanism ensured both reachability and obstacle avoidance under dynamic conditions (Chen et al., 2018). To address global path planning in uncertain environments, a Class-Ordered A* (COA*) algorithm was introduced in by Lim et al. where an optimal search tree is heuristically constructed on a weighted colored graph. Compared with the conventional A* algorithm, COA* shows improved performance in handling path uncertainty but remains dependent on explicit environment modeling, limiting its suitability for real-time dynamic applications (Lim and Tsiotras, 2021). In the field of robotic manipulator trajectory planning, population-based metaheuristic algorithms such as Genetic Algorithm (GA) and Particle Swarm Optimization (PSO) are widely used for their strong global search capability and flexibility in handling complex, multi-objective optimization problems; for example, Asma Seddaoui et al. proposed a GA-based path generator for controlled-floating space robots that leverages dynamic coupling between the manipulator and spacecraft base to safely reach grasping points while avoiding collisions and singularities, requiring only the Cartesian target position as input (Seddaoui and Saaj, 2021), and Özge Ekrem et al. developed a PSO-based trajectory planning method for a 6-DOF robotic arm using fifth-order polynomial interpolation to achieve obstacle-free, vibration-free, and time-optimized motion in joint space (Ekrem and Aksoy, 2023). However, these population-based metaheuristic approaches generally suffer from limited real-time performance, making them less suitable for dynamic obstacle avoidance in rapidly changing environments. Sampling-based algorithms, particularly Rapidly-exploring Random Trees (RRT), have gained widespread attention in manipulator motion planning due to their efficiency in high-dimensional spaces. Reference proposed RRT-Connect, which incrementally grows two trees from both start and goal configurations, successfully generating collision-free trajectories for a 6-DOF PUMA manipulator in three-dimensional workspaces (Kuffner and LaValle, 2000). However, sampling-based methods often suffer from excessive path length, poor smoothness, and suboptimal initial path quality due to their inherent randomness, necessitating extensive post-processing. To improve planning efficiency, Ding et al. developed the VSA-RRR algorithm, integrating an obstacle avoidance model for generating new sampling points and employing a workspace partitioning strategy to subdivide the manipulator's motion space into several subregions, thereby enhancing computational efficiency (Ding et al., 2025). Furthermore, Kong et al. investigated the trajectory tracking control problem of n-link robotic manipulators with dynamic uncertainties, time-varying output constraints, and input saturation. A finite-time control strategy was proposed to ensure end-effector collision avoidance in dynamic environments. However, this method primarily focuses on end-effector constraints without considering obstacle avoidance for the entire manipulator's state space, leaving room for further improvement in dynamic obstacle-rich environments (Kong et al., 2020). More recently, data-driven and skill-based planning frameworks have been explored to improve generalization across tasks. Liu et al. introduced DiffSkill, which extracts skill embeddings from offline datasets and leverages diffusion-based denoising to facilitate high-level policy learning for unseen tasks (Liu et al., 2024). While demonstrating strong generalization capability, such approaches typically rely on large-scale offline data and abstract skill representations, which may limit adaptability and fine-grained control in complex, rapidly changing environments.

Compared to traditional methods that often suffer from high computational complexity, poor adaptability, and reliance on precise environment models, Deep Reinforcement Learning (DRL) offers a more adaptive approach for robotic trajectory planning tasks. DRL agents can autonomously learn and adapt optimal policies through interactive sampling and policy optimization in complex environments without explicit modeling. Despite these advantages, DRL-based robotic trajectory planning in dynamic environments still encounters several key challenges that hinder its practical application and compromise performance robustness, including:

(1) Uncertainty and belief state instability in Partially Observable Markov Decision Process (POMDP) environments

(2) Multi-Objective Task Conflicts in Decision-Making

(3) Non-stationarity-induced challenges in policy convergence and generalization

To address these challenges, we propose a novel reinforcement learning framework for robotic manipulators operating in dynamic and complex environments. The framework combines an Adaptive Reward Shaping (ARS) strategy with a Trend Learning (TL) module and a Temporal-Difference error-based Q-value Knowledge Distillation (TDKDQ) mechanism. It is designed to improve adaptive obstacle avoidance and precise end-effector positioning under partially observable and non-stationary conditions. The effectiveness and robustness of the framework are validated through extensive simulation experiments in CoppeliaSim.

The main contributions of this paper are summarized as follows:

- A POMDP formulation is constructed to model dynamic obstacle avoidance and positioning control tasks for robotic manipulators in partially observable, dynamic environments, while simultaneously integrating a Trend Learning (TL) mechanism to capture temporal motion patterns and mitigate the effects of partial observability.
- A novel Adaptive Reward Shaping (ARS) method is proposed, which dynamically balances the conflicting objectives of dynamic obstacle avoidance and precise positioning in reinforcement learning for manipulators, enabling whole-body obstacle avoidance rather than limiting to end-effector safety.
- A Temporal-Difference error-based Q-value Knowledge Distillation (TDKDQ) mechanism is introduced to alleviate the non-stationarity caused by ARS and stabilize policy updates through a weighted soft Q-value estimation.
- Comprehensive CoppeliaSim simulations confirm that the proposed framework outperforms multiple baseline DRL algorithms in terms of effectiveness, robustness, and training stability under dynamic environmental conditions.

The remainder of this paper is organized as follows: Section 2 reviews related works. Section 3 First establishes a POMDP-based model to describe the dynamic obstacle avoidance and positioning control problem for robotic manipulators. Based on this model, presents the overall TL-ARS-TDKDQ framework, and details the design of its key components, including the TL method, ARS mechanism, and TDKDQ module. Section 4 describes the simulation platform setup, scene design, and algorithm parameter settings, followed by an analysis of the proposed framework's performance across various mainstream reinforcement learning algorithms under dynamic environments. Finally, Section 5 concludes the paper.

2. Related work

2.1. Deep reinforcement learning for robotic manipulator motion planning

In recent years, with the rapid advancement of deep learning, crucial conditions have been established for reinforcement learning (RL) to effectively tackle high-dimensional continuous state problems.

This has provided significant advantages in robotic high-dimensional continuous motion control. Numerous researchers worldwide have extensively studied deep reinforcement learning (DRL)-based robotic arm trajectory planning (Alegre et al., 2025).

Early efforts focused on model-free DRL methods for continuous control. Sangiovanni et al. employed a normalized advantage function (NAF) algorithm to extend Q-learning to continuous action spaces for robotic arm obstacle avoidance (Sangiovanni et al., 2018). While effective in controlled settings, the method assumes fixed obstacle motion directions and relies on handcrafted reward shaping, which limits its ability to cope with dynamically changing obstacle trajectories and partial observability commonly encountered in real-world robotic environments. To enhance environmental perception in cluttered workspaces, Zeng et al. utilized depth cameras to capture workspace information and trained robots using deep Q-networks (DQN) to perform collaborative pushing and grasping tasks (Zeng et al., 2018). Although the approach demonstrates promising performance in structured environments, its reliance on grid-based motion primitives and evaluation on limited object categories constrains the expressiveness and generalization capability of the learned policies. Beyond explicit reward engineering, several studies have explored learning reward functions directly from data. Wu et al. proposed a self-supervised framework to infer reward functions from high-dimensional robotic observations for estimating task progress (Wu et al., 2021). While the method yields superior manipulation policies and faster convergence compared to traditional baselines, it assumes monotonic task progression and is primarily evaluated on structured assembly tasks, limiting its applicability to non-monotonic or long-horizon manipulation scenarios. Similarly, Lim et al. introduced a data-efficient inverse reinforcement learning (IRL) approach based on L1-regularized sparse Gaussian Process prediction to recover reward functions from a small number of expert demonstrations (Lim et al., 2020). Despite its sample efficiency, the method requires access to diverse trajectory–reward pairs, which constrains scalability and practical deployment in high-dimensional robotic settings. Another line of work aims to improve learning efficiency and robustness through demonstration-based strategies. Rajeswaran et al. combined imitation learning with reinforcement learning by pretraining policies via behavior cloning and subsequently fine-tuning them using policy gradients (Rajeswaran et al., 2017). Although this strategy accelerates training and achieves robust performance across various manipulation tasks, its strong dependence on expert demonstrations reduces adaptability in complex and highly dynamic environments, particularly when demonstrations fail to capture evolving system dynamics. Safety-aware motion planning has also received increasing attention. Kiemel et al. developed a collision-aware planning framework that integrates a learned backup policy with a risk assessment module to enable safe and real-time trajectory generation under both deterministic and stochastic conditions (Kiemel et al., 2024). Nevertheless, frequent reliance on backup-policy interventions may disrupt the primary task policy, potentially affecting execution consistency in highly dynamic environments. Complementary approaches have investigated hybrid planning architectures and reward design strategies. Chen et al. designed a comprehensive reward function that integrates goal reaching and dynamic obstacle avoidance to enable real-time robotic arm planning (Chen et al., 2022), while Li et al. proposed a hybrid path planning framework that combines traditional planners with DRL to enhance planning performance (Li et al., 2021). Although these methods improve planning effectiveness, their reliance on manually designed components and heuristic planning structures reduces flexibility, limits transferability across tasks and environments, and increases system complexity.

2.2. Uncertainty and belief state instability in POMDP environments

The partially observable Markov decision process (POMDP) is a mathematical model that can describe a wide variety of sequential decision-making problems (Kochenderfer, 2015). Path planning for robotic manipulators in dynamic environments can be effectively formulated as POMDP, where both the robot and obstacle states are dynamic and partially observable. While traditional planners typically assume full observability and static conditions, these assumptions rarely hold in real-world scenarios, particularly in collaborative or industrial tasks involving moving obstacles (Lauri et al., 2022). Frequent, unpredictable changes in dynamic environments lead to belief state divergence and unstable policies, making it difficult to ensure safe, efficient, and reliable motion. Addressing these challenges requires scalable, real-time, and uncertainty-aware path planning frameworks tailored for high-dimensional POMDP problems.

Many efforts have been made to address decision biases in robot planning under dynamic and partially observable environments. Early sampling-based approximate planning methods have become a mainstream solution due to their flexibility and efficiency in large belief spaces. Thrun et al. employed particle filters for belief state estimation under uncertainty, providing a foundational approach for probabilistic state tracking in robotics (Thrun, 2002). Building on this, POMCP (Partially Observable Monte Carlo Planning) integrates Monte-Carlo belief updates with Monte-Carlo tree search, requiring only a black-box simulator and enabling scalable online planning in large, high-dimensional POMDPs (Silver and Veness, 2010). DESPOT (Determinized Sparse Partially Observable Tree) further improves planning stability by constructing sparse scenario-based decision trees with regularization (Somani et al., 2013). Nevertheless, these methods typically depend on extensive sampling or simulated rollouts to approximate belief evolution, which incurs substantial computational overhead and limits robustness and scalability in high-dimensional settings, particularly under rapidly changing dynamics or long-horizon tasks.

To further improve scalability, hierarchical and decomposition-based frameworks have been explored. Hierarchical point-based value iteration accelerates belief updates by exploiting structured state–action dependencies (Pineau et al., 2003), while multi-level solvers improve planning efficiency for nonlinear robotic systems (Hoerger et al., 2023). Despite these efficiency gains, such methods rely on predefined abstractions and hierarchical decompositions, which constrain adaptability in unstructured or rapidly evolving environments. Similarly, Saleem et al. proposed a hierarchical belief representation that first employs a coarse-grained policy to rapidly guide planning toward promising belief regions, followed by fine-grained planning to resolve high-dimensional uncertainties, leading to improved manipulator task success rates (Saleem et al., 2024). However, this coarse-to-fine hierarchical structure introduces additional system complexity and may suffer from error propagation between hierarchical levels, particularly when early-stage belief guidance is inaccurate.

Another line of work focuses on explicit risk management under uncertainty through chance-constrained motion planning (CCMP). Blackmore introduced an early CCMP framework that integrates probabilistic collision avoidance into trajectory optimization, enabling risk-aware planning for high-dimensional robotic systems (Blackmore et al., 2010). Building on this paradigm, p-Chekov extends CCMP to high-DOF robots by integrating LQG-based estimation, waypoint-level risk evaluation, and risk allocation, achieving high chance-constraint satisfaction under motion and sensing uncertainty (Dai et al., 2019). However, such approaches often rely on simplifying assumptions in uncertainty modeling, limited quadrature-based probability approximations, and conservative or suboptimal risk allocation strategies, which can result in overly cautious behaviors and reduced motion efficiency, particularly in highly nonlinear or contact-rich manipulation scenarios. Complementary to CCMP, online replanning and shielding-based methods enhance responsiveness by repeatedly updating belief-space plans or enforcing safety constraints during execution (Agha-mohammadi et al., 2018; Sheng et al., 2024). While effective in improving short-term safety, frequent replanning, additional statistical calibration or shielding overhead, and reliance on external safety mechanisms can lead to

policy oscillations and increased computational burden, and do not fundamentally resolve belief instability in long-horizon, continuous-state POMDP settings.

To improve decision consistency under partial observability, recurrent reinforcement learning algorithms such as DRQN (Hausknecht and Stone, 2015), R2D2 (Kapturowski et al., 2018), and Zheng et al.'s MK-A3C — a memory- and knowledge-based asynchronous advantage actor–critic integrating a GRU-based memory network — have been developed, enabling continuous control for robot navigation in unknown dynamic environments with moving obstacles (Zeng et al., 2019). Although effective for mitigating partial observability and accelerating task-specific convergence, temporal modeling in these methods remains largely implicit and tightly coupled to policy optimization, lacking explicit and decoupled trend abstraction. As a result, long-term temporal dependencies and motion evolution trends are not explicitly modeled, limiting robustness and stability in dynamic environments with non-Markovian state transitions.

2.3. Multi-objective task conflicts in decision-making

In general, robotic positioning under obstacle avoidance relies on evaluative functions comprising components for target achievement (Zhou et al., 2022), and for penalizing unsafe behaviors or constraint violations. While these components can be combined within a unified motion planning framework through weighted penalties, they are not always consistent and may even conflict (Zhou et al., 2023). In simple static environments, appropriately designed evaluative functions can effectively balance positioning accuracy and motion safety (Xie et al., 2021), often relying on the designer's intuition, experience, and prior knowledge (Shaker, 2016). However, with the growing demand for intelligent industrial production, many real-world scenarios involve dynamic and uncertain environments, such as human–robot collaboration tasks (Othman and Yang, 2023), where it becomes increasingly difficult to manually define appropriate evaluative criteria (Han et al., 2022; Escontrela et al., 2022). A lack of systematic understanding of the interactions between evaluative criteria and environment attributes can result in inefficiencies and increased system failure risks (Huang and Jin, 2022).

In terms of Gradient Conflict Resolution, Liu and Zhao employed an arbitration network (AR-SAC) to coordinate gradients from multiple tasks, effectively mitigating conflicts in multi-task reinforcement learning for robotic manipulators (Wang et al., 2025). Similarly, Kim et al. proposed CoMOGA (Constrained Multi-Objective Gradient Aggregator), which formulates multi-objective reinforcement learning as a constrained optimization problem and aggregates gradients through hierarchical constraints, successfully avoiding gradient conflicts while ensuring convergence performance (Kim et al., 2024) . To further address gradient interference issues, Sinha and Levine introduced DiGrad, a differential policy gradient method that separates shared and task-specific action components to reduce inter-task gradient interference in multi-objective robotic control (Dewangan et al., 2018). Despite their effectiveness in mitigating gradient conflicts during optimization, these approaches operate primarily at the gradient or action decomposition level and assume predefined task objectives, offering limited insight into how conflicting objectives should be structured or prioritized in dynamically changing environments.

In terms of Pareto-based Multi-objective Optimization, where solutions to multi-objective reinforcement learning (MORL) problems are commonly evaluated based on Pareto dominance criteria and the goal is to approximate the set of non-dominated solutions (Bai et al., 2023), Xu et al. proposed an efficient evolutionary learning algorithm for continuous robot control problems. This method extends a state-of-the-art reinforcement learning algorithm and incorporates a novel prediction model to effectively guide the learning process toward the Pareto frontier (Xu et al., 2020). In a related effort, Alegre et al. developed AMOR, a method that employs conditional policies to generate Pareto-optimal control strategies for dynamically weighted multi-objective tasks (Alegre et al., 2025). However, Pareto-based approaches typically incur high computational costs, require extensive exploration to approximate the Pareto front, and provide limited guidance for selecting a single operational policy in real-time robotic control scenarios.

From the perspective of reward function design, Tao et al. proposed an adaptive hierarchical reward mechanism (AHRM) that dynamically adjusts reward weights according to task phases, significantly improving learning efficiency in dexterous manipulation (Tao et al., 2022). Additionally, Berducci et al. introduced HPRS (Hierarchical Potential-based Reward Shaping), a structured reward shaping framework that prioritizes task objectives in layered potential functions, dynamically activating lower-priority rewards only after higher-priority objectives are achieved. This effectively mitigates reward conflicts and accelerates policy training (Berducci et al., 2021). While these methods reduce manual tuning effort and improve training stability, they still rely on handcrafted reward structures and task-specific heuristics, limiting scalability and generalization to unseen tasks or dynamic environments. To overcome the limited scalability of manual reward design and the poor generalization of demonstration-based methods, Cui et al. proposed GROVE — a general reward framework that combines LLMs and VLMs to iteratively optimize reward functions without manual engineering or task-specific demonstrations, enabling flexible adaptation across tasks (Cui et al., 2025). Although promising in reducing human effort, such approaches depend on external foundation models and offline reasoning loops, introducing additional system complexity and uncertainty in safety-critical robotic applications.

2.4. Non-stationarity-induced challenges in policy convergence and generalization

In dynamic environments, both the reward and state transition distributions may change over time in an exogenous manner — independently of the agent's policy — but within a constrained budget to limit excessive non-stationarity (Cheung et al., 2020). This intrinsic non-stationarity introduces significant challenges for reinforcement learning. Although existing DRL-based trajectory planning approaches have demonstrated promising performance in static environments, they still face substantial obstacles in dynamic, obstacle-laden scenarios. Frequent and unpredictable changes in environmental states complicate the design of stable and informative reward functions, undermining policy convergence and training stability. Moreover, increased environmental complexity amplifies the non-stationarity of state distributions and exacerbates task conflicts, making it difficult for policies to converge effectively. This situation corresponds to a non-stationary environment in which the agent must develop tailored dynamics models and policies to cope with distinct contexts and learning tasks.

Supervised meta-learning techniques (Finn et al., 2017) have recently been incorporated into reinforcement learning frameworks to facilitate swift policy adaptation in environments with changing dynamics (Nagabandi et al., 2018b). Nagabandi et al. introduced a model-based algorithm where a meta-learning technique is used to construct dynamics models capable of rapidly adapting to recent experiences (Nagabandi et al., 2018a)—either by updating the hidden state of a recurrent neural network (Duan et al., 2016), or by updating the model parameters via a small number of gradient steps (Finn et al., 2017). Similarly, Bing et al. proposed CEMRL, an efficient meta-reinforcement learning algorithm designed to handle non-stationary environments and broad task distributions (Bing et al., 2022). Despite their ability to accelerate adaptation, these meta-reinforcement learning approaches generally rely on past task experiences and assume that future dynamics resemble previously observed scenarios, which limits robustness and adaptability in highly non-stationary or previously unseen environments.

In terms of context- and risk-aware reinforcement learning methods, Touati et al. introduced the OPT-WLSVI algorithm, which leverages weighted least-squares value iteration with exponential weighting to gradually forget outdated data. This effectively handles contextual variations in non-stationary linear Markov decision processes, improving policy adaptability and stability (Touati and Vincent, 2020). Similarly, Hou et al. developed the Wasserstein Robust Advantage Actor–Critic (WRAAC) algorithm, which reformulates policy optimization as an iterative risk-sensitive task and links transition probability robustness with state robustness, thereby enhancing policy reliability in dynamic settings (Hou et al., 2020). However, they typically rely on assumptions such as linearity, bounded perturbations, or predefined feature representations, which can limit performance in highly nonlinear, unmodeled, or rapidly evolving scenarios, highlighting the need for approaches that maintain stable and flexible policy learning under more general dynamic conditions.

To mitigate catastrophic forgetting in multi-task and continual learning scenarios, continual reinforcement learning methods have been widely applied. Rolnick et al. proposed CLEAR, a replay-based method combining off-policy learning and behavior cloning within a hybrid experience replay mechanism. This approach balances learning stability with policy plasticity, significantly reducing forgetting in multi-task reinforcement learning environments (Li et al., 2024). Several distillation-based studies involve using one neural network as a reference or soft target for another. This technique enhances the training process by providing an additional auxiliary objective, with the trained network aiming to approximate or replicate the behavior of this target model. Early foundational works in knowledge distillation include model compression (Buciluǎ et al., 2006), teacher-student softmax matching (Hinton et al., 2015), and attention transfer (Zagoruyko and Komodakis, 2016), which collectively highlight the importance of effectively processing and transferring internal feature representations from a teacher to a student model. Building on these insights, modern reinforcement learning frameworks increasingly incorporate distillation mechanisms to stabilize training, guide policy learning, and transfer knowledge from expert or pretrained models, as demonstrated in recent reinforcement learning studies (Rusu et al., 2015; Zhang et al., 2022). While continual reinforcement learning approaches effectively mitigate catastrophic forgetting, they predominantly focus on preserving prior knowledge and lack mechanisms to explicitly balance stability with adaptability, limiting their responsiveness to novel tasks or abrupt environmental changes.

Furthermore, transfer reinforcement learning methods have also been explored to improve policy transferability under non-stationary conditions. Chai et al. proposed a re-weighted targeting procedure and designed the transfer deep Q*-learning framework. By strategically constructing transferable experience samples, this method enhances decision-making performance and provides theoretical convergence guarantees in dynamically changing environments (Chai et al., 2025). Nevertheless, the effectiveness of such methods is sensitive to task similarity and sample quality, and performance may degrade when applied to highly heterogeneous or rapidly evolving tasks.

3. Methodology

This section first formulates the dynamic obstacle avoidance and positioning control task of robotic manipulators as a Partially Observable Markov Decision Process (POMDP) under dynamic and partially observable environments. Based on this modeling, we construct the overall framework of the proposed TL-ARS-TDKDQ system. Then, we describe the proposed architecture, which consists of three key components: (1) Trend Learning for temporal pattern extraction and uncertainty mitigation, (2) Adaptive Reward Shaping for task-specific reward guidance and balancing between dynamic obstacle avoidance and precise positioning, and (3) Q-value knowledge distillation based on batch temporal-difference (TD) error for stable and efficient policy learning.

3.1. POMDP-based modeling for robotic manipulator dynamic obstacle avoidance and positioning control

To account for the dynamic and partially observable nature of the environment, the robotic manipulator control task is formulated as a POMDP, defined by the tuple:

$$(\mathcal{S}, \mathcal{A}, \mathcal{O}, \rho, \mathcal{O}b, \gamma, r) \tag{1}$$

where:

- $\mathcal{S}$ denotes the set of true environment states (unobservable to the agent).
- $\mathcal{A}$ is the continuous action space.
- $\mathcal{O}$ represents the observation space accessible to the agent.
- $\rho : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the state transition probability function.
- $\mathcal{O}b : \mathcal{S} \times \mathcal{A} \times \mathcal{O} \to [0, 1]$ is the observation probability function.
- $\gamma \in [0, 1]$ is the reward discount factor.
- $r : \mathcal{O} \times \mathcal{A} \to \mathbb{R}$ defines the reward function based on observations and actions.

In contrast to conventional POMDP solutions relying on belief states or observation-action histories, this framework adopts an observation sequence-based decision mechanism. At each timestep t, the agent receives an observation sequence:

$$h_t = (o_{t-n+1}, o_{t-n+2}, \dots, o_t) \tag{2}$$

The objective is to maximize the expected cumulative discounted reward:

$$v_\pi(h_t) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r(o_{t+k}, a_{t+k}) \,\middle|\, h_t \right] \tag{3}$$

The optimal policy π^* satisfies:

$$v_{\pi^*}(h_t) \geq v_\pi(h_t), \quad \forall h_t, \ \forall \pi. \tag{4}$$

Unlike standard POMDPs, we ignore explicit action histories and rely on sequential observations, as they sufficiently capture temporal dynamics through the continuous evolution of the robot, target, and obstacles, while improving computational efficiency.

To concretely implement this POMDP formulation, we next detail the design of the observation space, action space, and reward function.

(1) Observation space

At each timestep, the agent receives an observation vector $o_t \in \mathcal{O}$, comprising environmental and task-relevant features:

$$o_t = \left[\theta_i, P_{\text{TCP}}^{\text{position}}, P_{\text{TCP}}^{\text{posture}}, P_{\text{Target}}^{\text{position}}, P_{\text{Target}}^{\text{posture}}, E_{\text{Target}}^{\text{position}}, E_{\text{Target}}^{\text{posture}}, d_i^{\text{obs}}, H \right]. \tag{5}$$

A detailed description of the selected state variables is provided in Table 1.

(2) Action space

The agent selects continuous actions $\mathcal{A}$, representing incremental adjustments to joint angles:

$$a_t = \left[\Delta\theta_1, \Delta\theta_2, \Delta\theta_3, \Delta\theta_4, \Delta\theta_5, \Delta\theta_6, \Delta\theta_7 \right]. \tag{6}$$

(3) Reward function

The reward function is a weighted sum of three components, computed based on observations:

$$r_{\text{error}}(o_t) = -\left\| E_{\text{Target}}^{\text{position}} + E_{\text{Target}}^{\text{posture}} \right\|, \tag{7}$$

$$r_{\text{collision}}(o_t) = \sum_{i=1}^{6} \begin{cases} \frac{d_i^{\text{obs}}}{d_0} - 1, & \text{if } d_i^{\text{obs}} \leq d_0, \\ 0, & \text{otherwise,} \end{cases} \tag{8}$$

Table 1
Description of observation space elements.

Symbol	Description
θ_i	Joint angles of the manipulator's seven joints.
$P_{\mathrm{TCP}}^{\mathrm{position}}$	End-effector position in world coordinates.
$P_{\mathrm{TCP}}^{\mathrm{posture}}$	End-effector orientation in world coordinates.
$P_{\mathrm{Target}}^{\mathrm{position}}$	Target point position in world coordinates.
$P_{\mathrm{Target}}^{\mathrm{posture}}$	Target point orientation in world coordinates.
$E_{\mathrm{Target}}^{\mathrm{position}}$	Euclidean distance between TCP and target position.
$E_{\mathrm{Target}}^{\mathrm{posture}}$	Orientation deviation between TCP and target.
d_i^{obs}	Minimum distance between each potential collision module and obstacles.
H	Binary flag indicating whether TCP reached target pose (1: yes, 0: no).

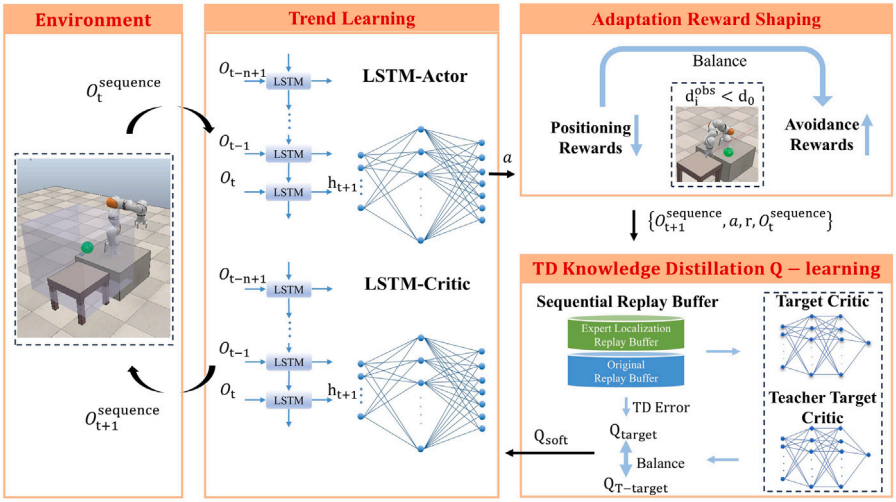


Fig. 1. The overall architecture of the proposed TL-ARS-TDKDQ framework for adaptive positioning control and obstacle avoidance in dynamic environments.

$$r_{\mathrm{keep}}(o_t) = \begin{cases} -1, & \text{if } P_{\mathrm{TCP}}^{\mathrm{position}} \notin p_{\mathrm{coverage}}^{\mathrm{position}}, \\ -0.5, & \text{if } P_{\mathrm{TCP}}^{\mathrm{position}} \in p_{\mathrm{coverage}}^{\mathrm{position}}, P_{\mathrm{TCP}}^{\mathrm{posture}} \notin p_{\mathrm{coverage}}^{\mathrm{posture}}, \\ 0, & \text{otherwise}, \end{cases} \tag{9}$$

Total reward:

$$r_t(o_t, a_t) = r_{\mathrm{error}}(o_t) + r_{\mathrm{collision}}(o_t) + r_{\mathrm{keep}}(o_t). \tag{10}$$

3.2. The TL-ARS-TDKDQ framework

This study addresses the challenges of robotic manipulator positioning control under dynamic obstacle avoidance in uncertain environments by proposing a deep reinforcement learning framework named TL-ARS-TDKDQ (see Fig. 1). The framework consists of three main components:

1. Trend Learning (TL), which enhances the Actor–Critic network with a temporal state encoding module to capture temporal dependencies in the robotic manipulator's sequential state data and optimize policy learning based on time-dependent motion trends;

2. Adaptive Reward Shaping (ARS), which dynamically balances the dual objectives of target localization and obstacle avoidance for the manipulator by adjusting rewards based on end-effector pose constraints and the frequency of boundary violations detected by the manipulator's various modules;

3. Temporal-Difference (TD) error-based Q-value Knowledge Distillation (TDKDQ) mechanism, which introduces a dynamic teacher network and adaptively balances the construction of soft target Q-values based on TD error, thereby mitigating policy oscillations and learning instability in the manipulator's control process caused by environmental non-stationarity.

The operation of the framework involves continuous perception of the robotic manipulator's state—including end-effector target positions, joint angles, and dynamic obstacle motion trends. The LSTM-enhanced actor–critic network extracts temporal features to support adaptive control decisions for complex manipulator positioning tasks under dynamic obstacle avoidance. Concurrently, The system continuously monitors the end-effector pose and the frequency of safety boundary violations by obstacles for potentially colliding manipulator modules, adaptively adjusting rewards to maintain a balance between target localization and obstacle avoidance to ensure smooth, balanced, and safe multi-objective control of the manipulator. To accelerate training, expert positioning control experiences relevant to robotic manipulators are integrated early, gradually transitioning to data generated by the manipulator's own interactions to enhance policy convergence. Finally, the TD error-based Q-value knowledge distillation stabilizes training by smoothing reward fluctuations and resolving conflicts among competing objectives typical in robotic manipulator control. Their coordinated integration within a unified RL framework enables adaptive, temporally aware, and robust control of robotic manipulators in complex, dynamic environments.

3.3. Trend learning

In this section, we address the challenge that positioning control for a robotic manipulator under dynamic obstacle avoidance is inherently a dynamic and complex motion process operating within a partially observable environment, Relying solely on the current observation for action decision-making and policy optimization may result in insufficient environmental representation, as it disregards valuable historical information and potential future variations. This limitation increases the risk of collisions or leads to overly conservative obstacle avoidance maneuvers in complex, dynamic workspaces, ultimately compromising both safety and task efficiency.

To overcome the inherent limitations of conventional observation representations in dynamic and partially observable environments, we

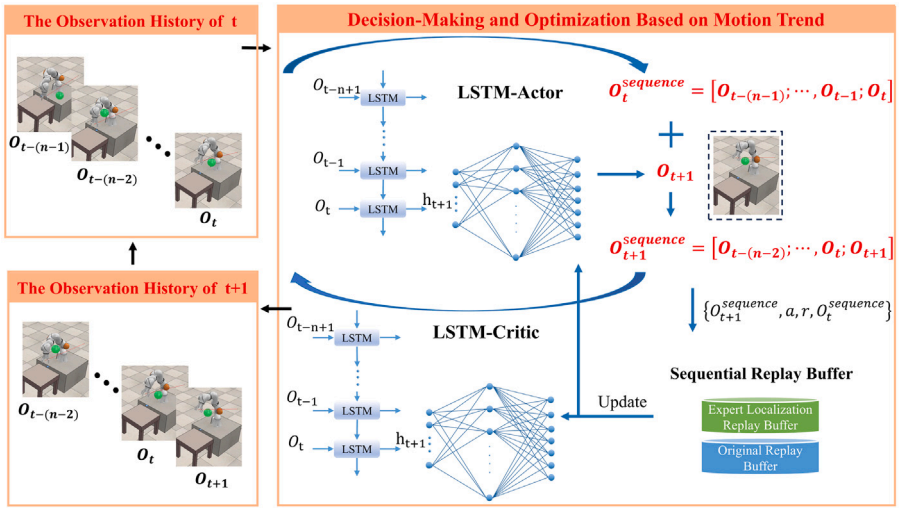


Fig. 2. The architecture of the proposed LSTM-based trend learning module for capturing temporal state representations in adaptive positioning control.

propose augmenting the decision-making process with trend-aware temporal observation encoding and sequential experience integration mechanisms (see Fig. 2). Specifically, to capture the latent temporal dependencies inherent in robotic manipulator motion within dynamic workspaces, Long Short-Term Memory (LSTM) networks are employed as temporal observation encoding and abstraction modules within both the Actor and Critic networks. These modules transform sequential observation trajectories into compact, trend-aware latent representations for downstream policy optimization and value estimation tasks.

The temporal encoders are co-optimized with their respective RL objectives, giving rise to a co-evolutionary learning dynamic where temporal abstraction and policy optimization mutually reinforce each other in the context of robotic manipulator control. Concretely, this dynamic is embodied in two LSTM-based temporal encoders— $\mathrm{LSTM}_{\theta^\pi}$ for the Actor and LSTM_{θ^Q} for the Critic—designed to process sequential observations of the manipulator's state. This ensures that both policy and value function learning capture temporally coherent motion patterns while remaining decoupled, enabling stable and efficient decision-making in dynamic, partially observable environments.

The Actor LSTM processes the observation sequence and produces a trend-aware latent state representation for policy decision-making:

$$z_t^\pi = \mathrm{LSTM}_{\theta^\pi}(h_t). \tag{11}$$

The policy network outputs a continuous action distribution conditioned on the trend-aware latent state z_t^π, modeled as a multivariate diagonal Gaussian. Its probability density function is given by:

$$\pi(a_t \mid z_t^\pi) = \frac{1}{\sqrt{(2\pi)^n |\Sigma_t|}} \exp\left(-\frac{1}{2}(a_t - \mu_t)^T \Sigma_t^{-1}(a_t - \mu_t)\right), \tag{12}$$

where $\mu_t \in \mathbb{R}^n$ and $\Sigma_t = \mathrm{diag}(\sigma_t^2)$ are the mean and covariance matrix output by the policy network given z_t^π, and n is the dimensionality of the action space.

Similarly, the Critic LSTM independently processes the same observation sequence to produce its own trend-aware latent representation for value estimation:

$$z_t^Q = \mathrm{LSTM}_{\theta^Q}(h_t). \tag{13}$$

The Critic network then estimates the state–action value function conditioned on this representation and the selected action:

$$Q(z_t^Q, a_t) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k r(o_{t+k}, a_{t+k}) \,\middle|\, z_t^Q, a_t\right]. \tag{14}$$

This dual-LSTM architecture allows both the Actor and Critic to effectively capture historical sequential dependencies while maintaining distinct temporal modeling capacities aligned with their respective objectives of policy optimization and value estimation in robotic manipulator control tasks. By decoupling the LSTM structures, the architecture prevents inter-network gradient interference, ensuring independent and stable updates of the policy and value functions. This design is particularly advantageous for robotic manipulators operating in dynamic, partially observable environments, where long-horizon temporal correlations and non-Markovian state transitions often undermine decision-making reliability.

To support this framework, we enhance the network optimization process by storing and sampling sequential observation experiences during the agent's interaction with the environment. The experience replay buffer D is structured as:

$$D = \left\{(h_t, a_t, r_t, h_{t+1})\right\}. \tag{15}$$

At each training iteration, minibatches of sequential transitions are sampled from D to perform off-policy updates for both the Actor and Critic networks, ensuring that temporal dependencies within observation sequences are preserved during policy and value function learning.

3.4. Task-guided reward shaping for balancing obstacle avoidance and positioning conflicts

The manipulator's positioning control and dynamic obstacle avoidance sub-tasks inherently conflict, especially when the manipulator and obstacles approach the target simultaneously. To dynamically balance these conflicting objectives, a task conflict-aware adaptive reward weighting mechanism is proposed. This mechanism assigns and adjusts reward weights for each collision module's avoidance sub-reward and the positioning control sub-reward based on real-time collision risk perception (see Fig. 3). The reward components are updated as follows:

$$t_i^c = \begin{cases} t_i^c + 1, & d_i^{\mathrm{obs}} \le d_0, \\ t_i^c, & d_i^{\mathrm{obs}} > d_0, \end{cases} \tag{16}$$

$$t_e = t - \sum_{i=1}^{6} t_i^c, \tag{17}$$

$$r_{\mathrm{error}} = -\left\| \mu_e \times t_e \, E_{\mathrm{Target}}^{\mathrm{position}} + E_{\mathrm{Target}}^{\mathrm{posture}} \right\|, \tag{18}$$

$$r_i^{\mathrm{collision}} = \begin{cases} \mu_i^c \times t_i^c \left(\frac{d_i^{\mathrm{obs}}}{d_0} - 1\right), & d_i^{\mathrm{obs}} \le d_0, \\ 0, & d_i^{\mathrm{obs}} > d_0, \end{cases} \tag{19}$$

$$r_{\mathrm{collision}} = \sum_{i=1}^{6} r_i^{\mathrm{collision}}. \tag{20}$$

Algorithm 1 TL Module: Trend-Aware Observation Encoding for Policy and Value Estimation

Require: Replay buffer $\mathcal{D}$; Actor network π_ϕ; Critic networks Q_θ
Require: Max episodes M, horizon T, batch size N; TL observation sequence length n
for episode = 1 to M **do**
  Initialize environment, obtain initial observation o_0
  Construct initial sequential observation buffer $h_0 = (o_0)$
  for timestep $t = 0$ to T **do** ▷ — Capture current and historical states for partially observable environment (Eq. 1–6) —
    Receive current observation o_t
    Update sequential observation buffer: $h_t = (o_{t-n+1}, \ldots, o_t)$
    ▷ — Trend Learning: Encode temporal patterns for Actor and Critic (Eq. 11–14) —
    Compute trend-aware latent for Actor (policy decision): $z_t^\pi = \mathrm{LSTM}_{\theta_\pi}(h_t)$
    Compute trend-aware latent for Critic (value estimation): $z_t^Q = \mathrm{LSTM}_{\theta_Q}(h_t)$
    ▷ — Critic estimates state-action value conditioned on trend-aware latent and selected action —
    $Q_t = Q_\theta(z_t^Q, a_t) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \middle| z_t^Q, a_t\right]$
    ▷ — Sample action conditioned on trend-aware latent state for safe and efficient motion —
    Select action $a_t \sim \pi_\phi(z_t^\pi)$
    Execute a_t; obtain reward r_t and next observation o_{t+1}
    Store sequential transition (h_t, a_t, r_t, h_{t+1}) in $\mathcal{D}$
  end for
end for

Here, t is the total weight, set based on total obstacle violation counts during training; t_e and t_i^c denote the dynamic reward weights for positioning and avoidance tasks, respectively, updated according to historical violation frequencies. μ_e and μ_i^c are the corresponding weight adjustment rates. It is worth noting that the adaptive reward shaping mechanism is inherently robust to the choice of weighting parameters. Empirically chosen initial values suffice, as the system automatically adjusts reward weights based on historical obstacle violation counts. Parameters such as the safety margin d_0, total weight t, and adjustment rates μ_e and μ_{ci} can be flexibly configured according to collision likelihood, dynamic obstacle characteristics, and training duration. This adaptivity ensures stable policy learning and effective balancing between positioning accuracy and collision avoidance across diverse scenarios (see Fig. 4).

To alleviate low data quality and inefficient exploration in early training, expert data collected from positioning control only (without obstacle avoidance) is incorporated. The sampling ratio between the current replay buffer L and the expert buffer P transitions gradually:

$$\begin{cases} B_t^L = \left\lfloor \frac{N}{2} + \frac{t}{T_s} \right\rfloor, \quad B_t^P = N - B_t^L, & 0 \le t \le \frac{NT_s}{2}, \\ B_t^L = N, \quad B_t^P = 0, & t > \frac{NT_s}{2}, \end{cases} \tag{21}$$

where B_t^L and B_t^P denote the batch sizes sampled from the current and expert buffers, respectively; N is the total batch size; and T_s is the decay step controlling the transition speed.

3.5. Q-value distillation based on batch TD error

In the non-stationary environment induced by Adaptive Reward Shaping (ARS) for robotic manipulator control tasks, conventional target network-based Q-value estimations are prone to oscillations and instability due to frequent policy updates and dynamic environment feedback. To address this issue, this study proposes the TDKDQ module, which introduces a soft Q-value fusion mechanism between a teacher target network and the conventional target network, effectively mitigating such instability in robotic manipulator policy training (see Fig. 5).

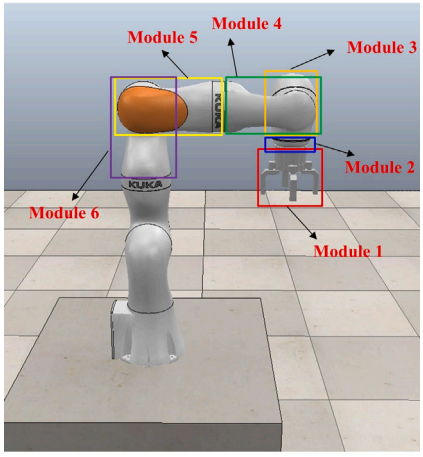


Fig. 3. The figure illustrates the potential collision modules of the robotic arm within the spatial range of obstacle movement. These modules perform closest-distance detection with obstacles at each step of reinforcement learning to obtain partial state information.

Algorithm 2 ARS Module: Adaptive Reward Shaping

Require: ARS coefficients $\{\mu_e, \mu_i^c\}_{i=1}^6$, safe distance d_0
for each timestep t in episode **do** ▷ — Adaptive Reward Shaping for Balancing Positioning and Obstacle Avoidance (Eq. 7–10, 16–20) —
  for $i = 1$ to 6 **do**
    if $d_i^{\text{obs}} \le d_0$ **then**
      $t_i^c \leftarrow t_i^c + 1$ ▷ Increase avoidance weight for module i
    end if
  end for
  $t^e = t - \sum_{i=1}^6 t_i^c$
  $r_{\text{error}} = -\mu_e \cdot t^e \cdot (E_{\text{Target}}^{\text{position}} + E_{\text{Target}}^{\text{posture}})$
  $r_{\text{collision}} = 0$
  for $i = 1$ to 6 **do**
    if $d_i^{\text{obs}} \le d_0$ **then**
      $r_{\text{collision}}^i = \mu_i^c \cdot t_i^c \cdot \left(\frac{d_i^{\text{obs}}}{d_0} - 1\right)$
      $r_{\text{collision}} \leftarrow r_{\text{collision}} + r_{\text{collision}}^i$
    end if
  end for
  if $P_{\text{TCP}}^{\text{pos}} \notin C_{\text{pos}}$ **then**
    $r_{\text{keep}} = -1$
  else if $P_{\text{TCP}}^{\text{post}} \notin C_{\text{post}}$ **then**
    $r_{\text{keep}} = -0.5$
  else
    $r_{\text{keep}} = 0$
  end if
  $r_t' = r_{\text{error}} + r_{\text{collision}} + r_{\text{keep}}$
end for

Specifically, TDKDQ incorporates an additional teacher target network, structurally identical to the main critic network, which periodically clones parameters from the main critic. This mechanism ensures the teacher target network retains more stable historical knowledge, serving as an informative reference for guiding the main critic network in an online knowledge distillation manner, rather than transferring knowledge from a model trained on a different task. During training, based on the Temporal-Difference (TD) error computed from current experience samples, the fusion ratio between the teacher target Q-value and the conventional target Q-value is dynamically adjusted, forming a soft target Q-value for training the main critic network. When the TD error is large, the model relies more on the teacher target network to utilize stable prior knowledge; when the TD error is small, it places

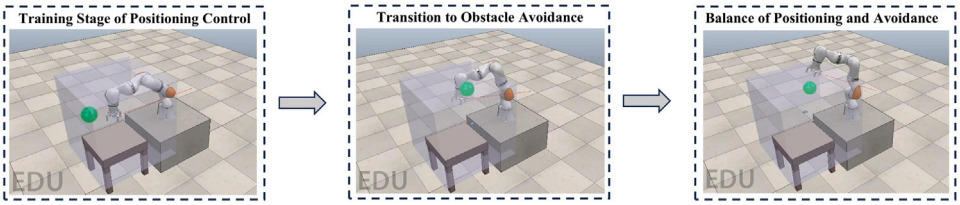


Fig. 4. This figure illustrates the adaptive task transition and balancing process achieved through reward shaping. The transition is guided by the frequency of obstacles crossing the safety boundaries of each robotic arm module in the real environment, progressively shifting the learning focus from positioning control to positioning control with dynamic obstacle avoidance, ensuring a balanced and efficient policy optimization process.

greater emphasis on the current target network to maintain sensitivity to the latest environment feedback.

This TD error-guided adaptive weighting mechanism effectively implements the distillation process by dynamically balancing reliance on stable prior knowledge and current environment feedback, mitigating value function fluctuations caused by reward weight adjustments, while avoiding excessive smoothing that could delay policy updates. Consequently, TDKDQ achieves a balanced trade-off between stability and adaptability, significantly enhancing the robustness and convergence efficiency of robotic manipulator policy training in non-stationary environments.

Taking the combination of the Soft Actor–Critic (SAC) algorithm and TDKDQ as an example under a sequence-based POMDP formulation, the optimal policy is defined by:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{h_t\sim D}\left[\sum_{t=0}^{\infty}\gamma^t\left(R(h_t,a_t)+\alpha H(\pi(\cdot\mid h_t))\right)\right]. \tag{22}$$

Here, D is the replay buffer containing observation sequences h_t, and α is the temperature parameter, decayed over time as:

$$\alpha = e^{-\lambda t}\alpha_0. \tag{23}$$

The clipped double Q-learning structure adopts six Q-networks (Fujimoto et al., 2018). The target Q-value for a sequence-based decision is computed as:

$$Q_{\text{target}}(h_t,a_t) = r_t + \gamma\,\mathbb{E}_{a_{t+1}\sim\pi(\cdot|h_{t+1})}\left[\min_{j=1,2} Q_{\text{target},j}(h_{t+1},a_{t+1}) - \alpha\log\pi(a_{t+1}\mid h_{t+1})\right]. \tag{24}$$

Similarly, the teacher target Q-value is given by:

$$Q_{\text{T-target}}(h_t,a_t) = r_t + \gamma\,\mathbb{E}_{a_{t+1}\sim\pi(\cdot|h_{t+1})}\left[\min_{j=1,2} Q_{\text{T-target},j}(h_{t+1},a_{t+1}) - \alpha\log\pi(a_{t+1}\mid h_{t+1})\right]. \tag{25}$$

To address non-stationary environments, the soft target Q-value integrates both target and teacher target Q-values, weighted by the batch-normalized TD error:

$$Q_{\text{soft}}(h_t,a_t) = \beta E_{\text{TD}}^{\text{norm}} Q_{\text{T-target}}(h_t,a_t) + \left(1-\beta E_{\text{TD}}^{\text{norm}}\right) Q_{\text{target}}(h_t,a_t). \tag{26}$$

where $E_{\text{TD}}^{\text{norm}} \in [0, 0.2]$.

The critic loss is then computed as:

$$J(Q) = \mathbb{E}_{(h_t,a_t,r_t,h_{t+1})\sim D}\left[\left(Q_1(h_t,a_t)-Q_{\text{soft}}(h_t,a_t)\right)^2 + \left(Q_2(h_t,a_t)-Q_{\text{soft}}(h_t,a_t)\right)^2\right]. \tag{27}$$

The actor loss follows the maximum entropy objective (Haarnoja et al., 2018):

$$J(\pi) = \mathbb{E}_{h_t\sim D}\left[\mathbb{E}_{a_t\sim\pi(\cdot|h_t)}\left[\alpha\log\pi(a_t\mid h_t) - \min_{j=1,2} Q_j(h_t,a_t)\right]\right]. \tag{28}$$

Soft updates for the target networks and parameter cloning for the teacher target critic network are performed as follows:

$$\theta'_{j=1,2} \leftarrow \tau\theta_{j=1,2} + (1-\tau)\theta'_{j=1,2}, \tag{29}$$

$$\phi' \leftarrow \tau\phi + (1-\tau)\phi', \tag{30}$$

$$\theta''_{j=1,2} \leftarrow \theta_{j=1,2}. \tag{31}$$

Here, θ_j denotes the main critic network parameters (for $j = 1, 2$), θ'_j are the traditional target critic network parameters, θ''_j are the teacher target critic network parameters obtained by cloning the main critic parameters, ϕ and ϕ' are the actor network and its target network parameters, and τ is the soft update coefficient.

Finally, to improve stability, the actor is updated less frequently than the critic:

$$f_{\text{Actor}}^{\text{cur}} = \frac{f_{\text{Critic}}^{\text{cur}}}{n_f}. \tag{32}$$

Here, n_f denotes the actor update frequency relative to the critic.

Algorithm 3 TDKDQ Module: TD-Error-Guided Soft Q-Value Fusion

Require: TDKDQ update interval n_f; fusion coefficient β; Teacher target update interval T_f
Require: Critic networks Q_θ; Target Critic networks $Q_{\theta'}$; Teacher Critic networks $Q_{\theta''}$
for each training iteration **do**
  Sample minibatch $\mathcal{B}$ from $\mathcal{D}$ and expert buffer $\mathcal{P}$
  ▷ — Compute soft target Q-value based on batch-normalized TD error (Eq. 24–27) —
  Compute standard target Q-value: $Q_{\text{target}}(h_t,a_t)$ using Eq. (24)
  Compute teacher target Q-value: $Q_{\text{T-target}}(h_t,a_t)$ using Eq. (25)
  Compute batch TD errors: $\delta_t = Q_{\text{target}}(h_t,a_t) - Q_\theta(h_t,a_t)$
  Normalize TD error within batch: $E_{\text{TD}}^{\text{norm}} = \text{NormalizeBatch}(|\delta_t|)$, $E_{\text{TD}}^{\text{norm}} \in [0, 0.2]$
  Compute soft fused target: $Q_{\text{soft}} = \beta E_{\text{TD}}^{\text{norm}} Q_{\text{T-target}} + (1-\beta E_{\text{TD}}^{\text{norm}}) Q_{\text{target}}$
  Critic loss: $J(Q) = \frac{1}{|\mathcal{B}|}\sum\left[(Q_1-Q_{\text{soft}})^2 + (Q_2-Q_{\text{soft}})^2\right]$
  Update critic parameters θ
  if update step $\mod n_f = 0$ **then**
    Actor loss: $J(\pi) = \mathbb{E}_{h_t,a_t}\left[\alpha\log\pi(a_t|h_t) - \min_{j=1,2} Q_j(h_t,a_t)\right]$
    Update actor parameters ϕ
  end if
  ▷ — Soft update for target networks (Eq. 29–30) —
  $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$
  $\phi' \leftarrow \tau\phi + (1-\tau)\phi'$
  if update episode $\mod T_f = 0$ **then** ▷ — Periodically clone main critic parameters to teacher critic (Eq. 31) —
    $\theta'' \leftarrow \theta$
  end if
end for

4. Experiments and results

This section presents the experimental platform setup, parameter configurations, and the performance evaluation of the proposed method through extensive experiments and ablation studies.

4.1. Experimental platform and parameter settings

The experimental simulation platform was established based on CoppeliaSim Edu version 4.6. The proposed algorithm was implemented using the Python programming language with the PyTorch

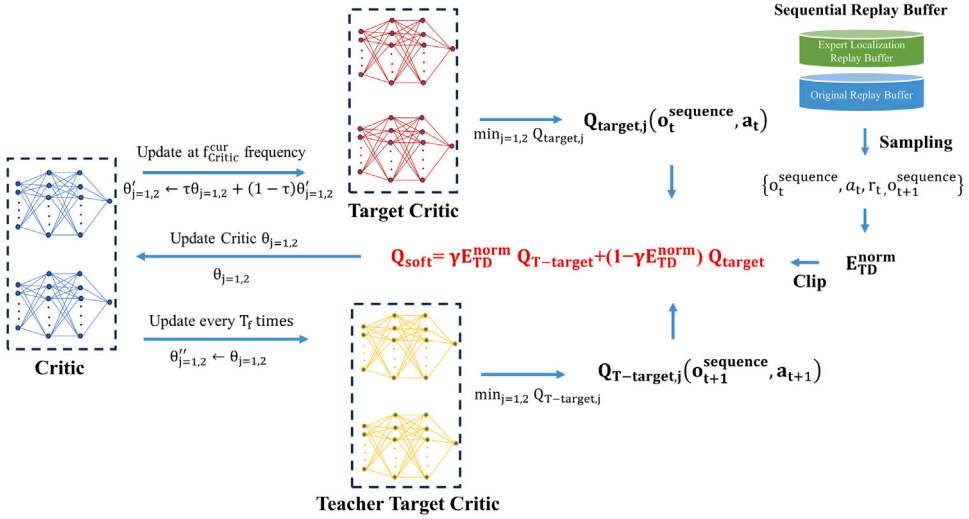


Fig. 5. This figure illustrates the update and training process for generating soft target Q-values by balancing the target Q-network and the teacher target Q-network, based on batch-normalized TD errors as evaluation indicators for non-stationary environments. During optimization, the past knowledge introduced by the teacher target Q-network effectively mitigates the risk of training collapse caused by instability in non-stationary reinforcement learning scenarios.

framework. Through the Remote API provided by CoppeliaSim, network communication between Python and the CoppeliaSim platform was realized. By parsing feedback data from the CoppeliaSim Edu simulation platform, the state information of the mobile manipulator was acquired in real-time.

To assess the runtime efficiency of the proposed method, we measured the per-step inference time of the trained policy on a workstation equipped with an 11th Gen Intel Core i9-11900K CPU and an NVIDIA Quadro RTX 4000 GPU. The execution time per control cycle ranges from 3 to 8 ms, demonstrating that the model is capable of real-time control for industrial robotic applications.

The dynamic obstacle avoidance and position control simulation environment is illustrated in Fig. 6. A simulated seven-degree-of-freedom (7-DOF) KUKA robotic manipulator was constructed within CoppeliaSim. Compared to traditional manipulator models, this configuration provides enhanced flexibility, enabling it to better adapt to complex environments involving real-time dynamic obstacle avoidance. The manipulator's operational workspace is defined as an orange transparent rectangular area with dimensions of 0.3 m × 0.15 m × 0.1 m. Within this workspace, target points (represented as small blue squares) are randomly generated. The manipulator's task is to start from an initial state and accurately achieve the specified position and orientation control at the target point using the TCP (Tool Center Point) of its end effector, while avoiding dynamic obstacles and maintaining the target posture for a continuous period.

To simulate dynamic obstacles, a green spherical object with a radius of 0.075 m moves at a constant speed of 0.22 m/s near the target point, representing the dynamic obstacle's motion. The obstacle moves randomly within a cubic blue transparent space with a side length of 0.7 m centered on the target point, and randomly changes direction upon colliding with the boundaries. The manipulator is controlled via network communication between Python and CoppeliaSim, while the motion of the dynamic obstacle is managed through an internal CoppeliaSim script.

The key parameters involved in the experiment are listed in Table 2. These include hyperparameters for training iterations, learning rates, experience replay settings, network architecture specifications, action and state space configurations, as well as reward shaping and control constraints.

4.2. Performance of TL-ARS-TDKDQ algorithm

To validate the generality and effectiveness of the proposed TL-ARS-TDKDQ framework when integrated with different policy gradient-based reinforcement learning algorithms, it was incorporated respectively into three mainstream continuous action space algorithms: Deep Deterministic Policy Gradient (DDPG), Twin Delayed Deep Deterministic Policy Gradient (TD3), and Soft Actor–Critic (SAC), as well as three additional representative baselines: Conservative Q-Learning (CQL), Demonstration-Augmented Policy Gradient with behavior cloning (DAPG), and Truncated Quantile Critics (TQC). These integrated variants were evaluated within the dynamic obstacle avoidance and position control task scenario for a mobile manipulator.

A comprehensive evaluation was conducted using multiple performance metrics, including: (i) the number of episodes required for reward convergence (N_{conv}), (ii) the average episode reward after convergence ($\overline{R}_{\text{conv}}$), (iii) the average number of time steps required to complete the motion planning task after convergence ($\overline{T}_{\text{task}}$), (iv) the standard deviation of episode rewards after convergence (σ^R_{conv}), and (v) the task success rate (P_{rate}).

The convergence criterion was defined as follows: if the absolute difference between the average rewards over the most recent 50 episodes and the preceding 50 episodes is less than a threshold δ, and the average reward exceeds a minimum threshold $R_{\min}$, the policy is considered converged. Formally,

$$\left| \frac{1}{50} \sum_{i=n-49}^{n} r_i - \frac{1}{50} \sum_{i=n-99}^{n-50} r_i \right| < \delta, \quad \frac{1}{50} \sum_{i=n-49}^{n} r_i > R_{\min}, \quad n > 100, \tag{33}$$

where $\delta = 0.1$ and $R_{\min} = -50$ in our experiments.

The success rate was evaluated by conducting five rounds of experiments with each trained policy model, each round consisting of 200 independent tests. A test was considered successful if the manipulator completed the task without any collisions with dynamic obstacles, and maintained the target pose continuously for a specific number of steps within the defined tolerances ($|\Delta d| \leq 0.025$ m, $|\Delta\theta| \leq 0.087$ rad, $T = 5$ steps). The final success rate was calculated as the average success rate across the five rounds of experiments (see Figs. 7–11).

The detailed experimental results are presented in Table 3.

The experimental results demonstrate that the proposed TL-ARS-TDKDQ framework exhibits robust convergence and strong task performance across multiple mainstream reinforcement learning algorithms.

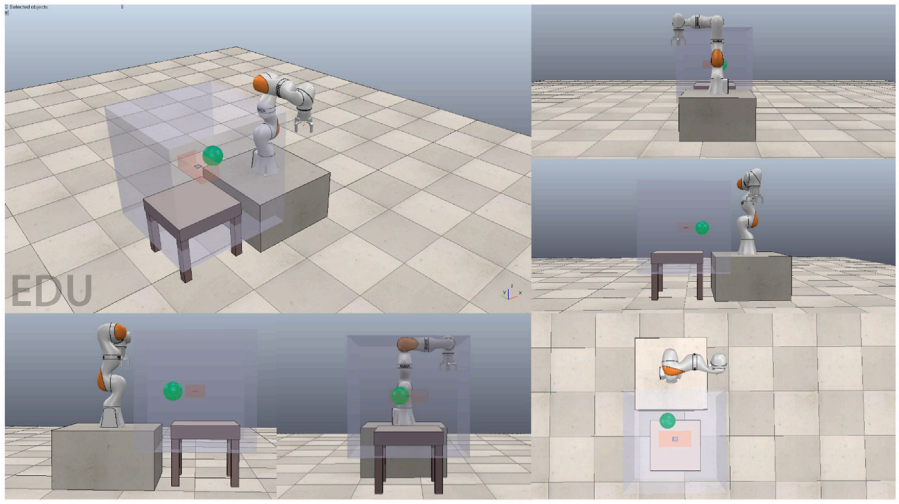


Fig. 6. Adaptive motion planning simulation environment for a robotic manipulator in a dynamic environment based on CoppeliaSim.

Table 2
Experimental parameter settings.

Parameter	Value
Training episodes M	3000.
Max steps per episode T	300.
Discount factor γ	0.97.
Critic learning rate	0.0003.
Actor learning rate	0.0003.
Replay buffer size	1×10^4.
Mini-batch size N	128.
Decay steps T_s	2000.
Numerical stability constant	1×10^{-6}.
Gradient-safe clipping interval for policy gradient term $\nabla_\pi \log(1-\pi^2)$	[0, 1].
Actor update frequency relative to the Critic n_f	2.
Soft update factor τ	0.01.
Teacher target critic update interval T_f	50.
Weighting factor β	3.
Batch-normalized TD error truncation range $E_{\mathrm{TD}}^{\mathrm{norm}}$	[0, 0.2].
Initial exploration noise range	[−4, 4].
Initial temperature coefficient α_0	0.1.
Decay factor λ	0.0016.
Log std truncation range	(−20, 2).
Critic update frequency $f_{\mathrm{Critic}}^{\mathrm{cur}}$	1.
Actor update frequency $f_{\mathrm{Actor}}^{\mathrm{cur}}$	1/2.
LSTM sequence length	3.
LSTM hidden size	256.
Number of LSTM layers	1.
Fully connected layers	2.
Hidden layer size	512.
Action space dimension $\dim(a_t)$	7.
State space dimension $\dim(s_t)$	29.
Action increment range $\Delta\theta_i$ (rad)	[−10, 10].
Position error tolerance Δd (m)	[−0.025, 0.025].
Orientation error tolerance $\Delta\theta$ (rad)	[−0.087, 0.087].
Pose maintenance steps n	5.
Safety boundary d_0 (m)	0.05.
Total weight sum t	1×10^5.
Target reward shaping rate μ_e	3×10^{-4}.
Obstacle reward shaping rates μ_i^c (Joints 1, 7)	3×10^{-2}.
Obstacle reward shaping rates μ_i^c (Joints 2-6)	6×10^{-2}.
Conservative regularization coefficient α_{CQL}	1.0.
Demonstration weight λ_{BC} (training episode i)	$1.0 \cdot \exp(-0.01\, i)$.
Demonstration batch size N_{demo}	64.
Number of quantiles per critic N_q	10.
Number of dropped quantiles N_{drop}	3.
Quantile Huber loss parameter κ	1.0.

Compared with conventional continuous-control RL methods, our approach can adaptively learn in complex dynamic scenarios involving highly conflicting tasks of precise positioning and dynamic obstacle avoidance, where other algorithms fail to achieve effective learning or convergence. From the loss perspective, driven by the dominance of the positioning reward and the high proportion of positioning expert experience samples early in training, the Critic loss starts low. As rewards become non-stationary and expert samples diminish, the

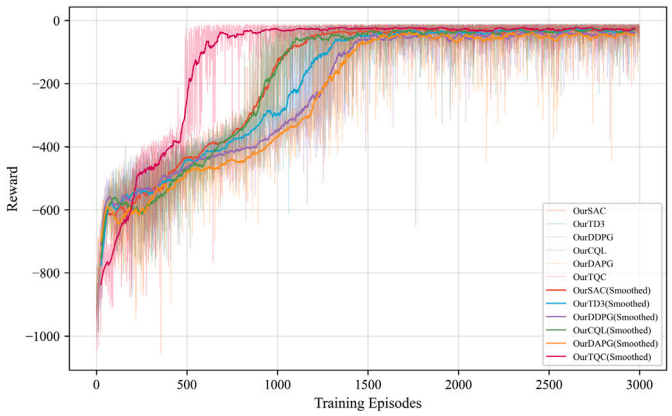


Fig. 7. Reward convergence trends of algorithms integrated with the TL-ARS-TDKDQ framework in adaptive motion planning for robotic manipulators in dynamic environments.

Table 3
Performance comparison of the TL-ARS-TDKDQ framework integrated into different policy gradient algorithms.

Method	N_{conv}	$\overline{R}_{\text{conv}}$	$\overline{T}_{\text{task}}$	σ^{R}_{conv}	S_{rate} (%)
Our SAC	1373	−32.83	28.64	28.42	94.5
Our TD3	1419	−37.03	33.21	29.22	92.8
Our DDPG	1777	−46.05	42.57	42.29	91.3
Our CQL	1184	−37.58	31.78	39.40	92.7
Our DAPG	1858	−50.88	45.24	51.34	91.0
Our TQC	1033	−27.16	22.13	19.86	91.4

loss converges with low-amplitude oscillations. CQL yields higher loss due to regularization, whereas TQC achieves smaller loss variance via quantile-based estimation and truncation. From the reward perspective, TL-ARS-TDKDQ-TQC achieved the fastest convergence (1033 episodes) and the most stable reward evolution, owing to its quantile-based critic design, which reduces reward variance but slightly limits the task success rate (91.4%) due to conservative value estimation. TL-ARS-TDKDQ-SAC obtained the highest success rate (94.5%) with consistent convergence, while TL-ARS-TDKDQ-TD3 offered a good balance between learning speed and policy stability. For TL-ARS-TDKDQ-DAPG, although behavior cloning with expert data was applied during the early-stage positioning tasks, the task was simple and the effect was limited. During the transition to dynamic obstacle avoidance with positioning control, the imitation influence slightly constrained the learning of obstacle avoidance. Notably, in our implementation of TL-ARS-TDKDQ-DDPG, a double-Q network was incorporated to mitigate overestimation. Despite this, it converged more slowly (1777 episodes) and achieved lower reward (−46.05) and longer task duration (42.57 s). TL-ARS-TDKDQ-CQL benefited from conservative Q-regularization, improving training stability, but its overall performance remained moderate (converged in 1184 episodes, reward −37.58, task duration 31.78). Overall, these results confirm that the TL-ARS-TDKDQ framework enhances stability, adaptability, and convergence efficiency across different algorithm integrations, while the characteristics of each integration reflect their inherent design trade-offs.

4.2.1. Ablation study on trend learning

In the ablation studies of the Trend Learning (TL) module for robotic manipulators operating in dynamic and uncertain environments, we explicitly investigate two complementary settings to assess the contribution of the LSTM: (i) temporal context ablation and (ii) structural ablation. For temporal context ablation, the LSTM sequence length is restricted to $n = 1$, which inhibits the model's ability to exploit temporal dependencies while preserving the original LSTM cell structure and gating operations. This setting enables an examination of temporal context effects without introducing architectural changes or parameter-scale discrepancies. In contrast, using a longer input sequence with $n = 3$ allows the LSTM to leverage short-term temporal context. The results demonstrate that incorporating temporal sequences significantly reduces decision biases induced by environmental uncertainty. For structural ablation, the LSTM module is entirely removed from the TL module. Compared to the temporal context ablation setting, this structural removal exhibits a more evident performance decline. The ablation analysis suggests that trend learning helps compensate for incomplete state observability in POMDPs, thereby improving policy robustness in dynamic environments (see Figs. 12–15).

Furthermore, we investigated the impact of different temporal sequence lengths on the control performance of the robotic manipulator. The experimental results show that incorporating state information from the past 2 to 3 time steps to construct temporal sequences yields the best performance for the manipulator's policy. However, as the sequence length continues to increase, the control performance begins to degrade (see Figs. 16 and 17). We attribute this phenomenon to the characteristics of the operating environment: since obstacles move at a constant linear velocity and only change direction randomly upon reaching motion boundaries, 2 to 3 consecutive states are sufficient for the robotic manipulator to infer movement trends and make reliable decisions. Excessively long sequences introduce redundant information and may exacerbate long-term dependency issues, such as gradient vanishing or information dilution, thereby impeding effective policy learning for the manipulator. In addition, overly long sequences may incorporate irrelevant or outdated information, weakening the model's sensitivity to current critical dynamic features in the manipulator's workspace. Since the setting without the Trend Learning (TL) module, as well as certain sequence length configurations under the TL-enabled setting, failed to achieve convergence within the given training episodes, we instead adopted three alternative indicators to evaluate the manipulator's performance: the mean reward over the final 500 episodes ($\overline{R}_{500}$), the corresponding standard deviation ($\sigma_{R_{500}}$), and the task success rate (S_{rate}). The results are presented in Table 4.

4.2.2. Ablation study on adaptive reward shaping and generalization performance

To verify the guiding and balancing effects of the adaptive weight adjustment mechanism in positioning control tasks for robotic manipulators under dynamic obstacle avoidance conditions, we directly assigned the balanced parameters obtained from the adaptive adjustment mechanism to the corresponding parameters of each potential collision module of the manipulator. The manipulator was then trained for the same number of episodes. This experiment aims to demonstrate how the gradual transition from simple to complex environments guides and balances positioning control tasks specifically for robotic manipulators in dynamic obstacle avoidance scenarios. The experimental results indicate that, compared to conventional manually assigned reward–penalty functions, the adaptive reward adjustment mechanism effectively: (1) Dynamically adjusts reward and penalty terms during training, enabling the manipulator's control agent to autonomously find an optimal trade-off without the need for laborious manual orthogonal experiments to tune weight parameters. (2) Facilitates a gradual transition from simple to complex task scenarios, which significantly accelerates training convergence and enhances the agent's ability to learn and adapt in complex dynamic environments. By progressively adjusting the reward and penalty mechanisms, the manipulator achieves a well-balanced performance between dynamic obstacle avoidance and precise positioning control, thereby significantly enhancing its overall task effectiveness (see Figs. 18–21).

To further validate the generalization capability of the proposed method, we conducted a comparative test between a model trained with the adaptive reward shaping (ARS) mechanism and a model trained with preset fixed weight parameters. Both models were evaluated 200 episodes. In testing, the task requirements were made

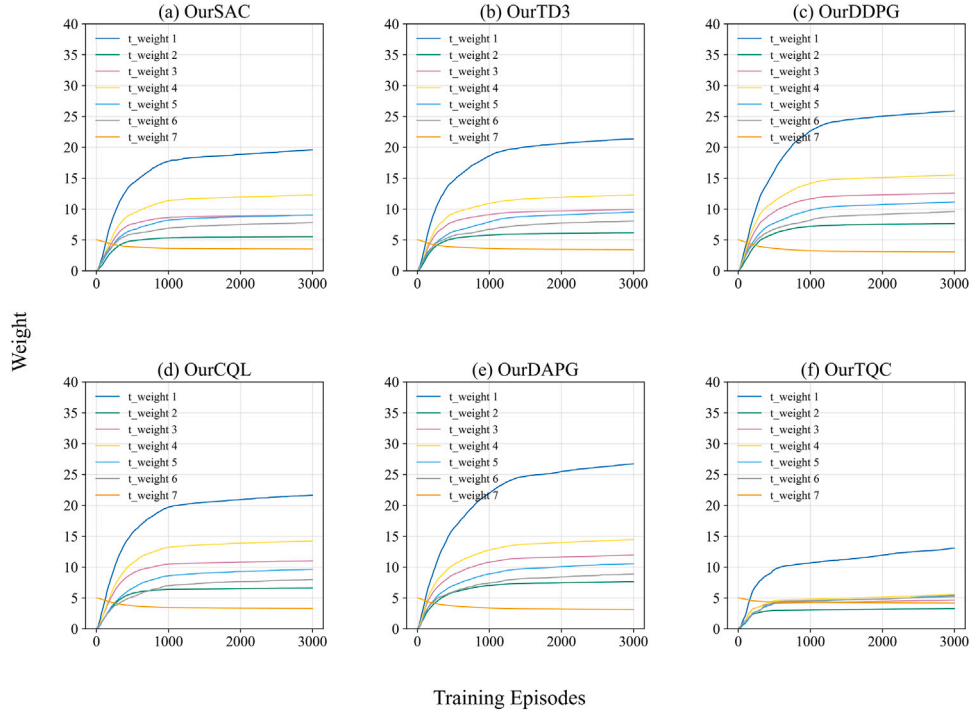


Fig. 8. Evolution curves of the reward weights for potential collision modules and the end-effector module during training under the TL-ARS-TDKDQ framework in adaptive motion planning for robotic manipulators.

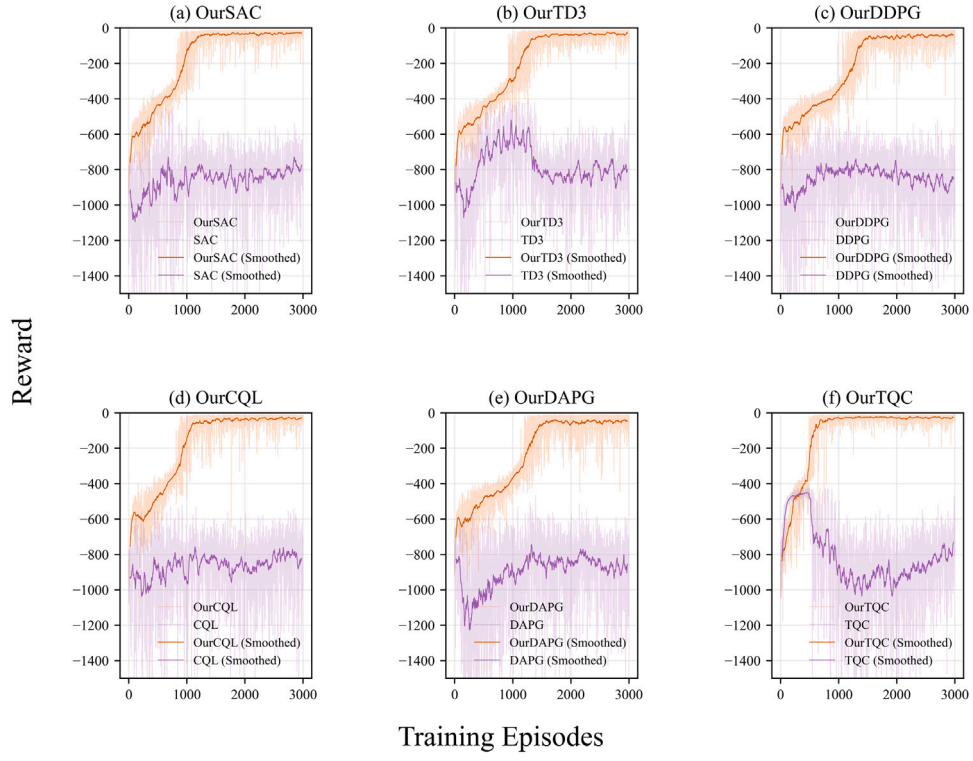


Fig. 9. Comparative analysis of reward convergence trends between algorithms integrated with the TL-ARS-TDKDQ framework and baseline algorithms in adaptive motion planning for robotic manipulators operating in dynamic environments.

more stringent than during training—specifically, the position accuracy threshold was tightened to 0.015 m, and the duration for maintaining the target position was extended to 9 time steps. These adjustments increased the collision risk, thereby intensifying the challenge of the testing scenarios. Under these heightened demands, the manipulator controlled by the ARS-trained model consistently demonstrated strong task completion ability and stability, reflecting robust generalization under increased task complexity. Conversely, the manipulator governed by the fixed-weight model showed a noticeable performance decline, with a significantly lower success rate, highlighting its limited

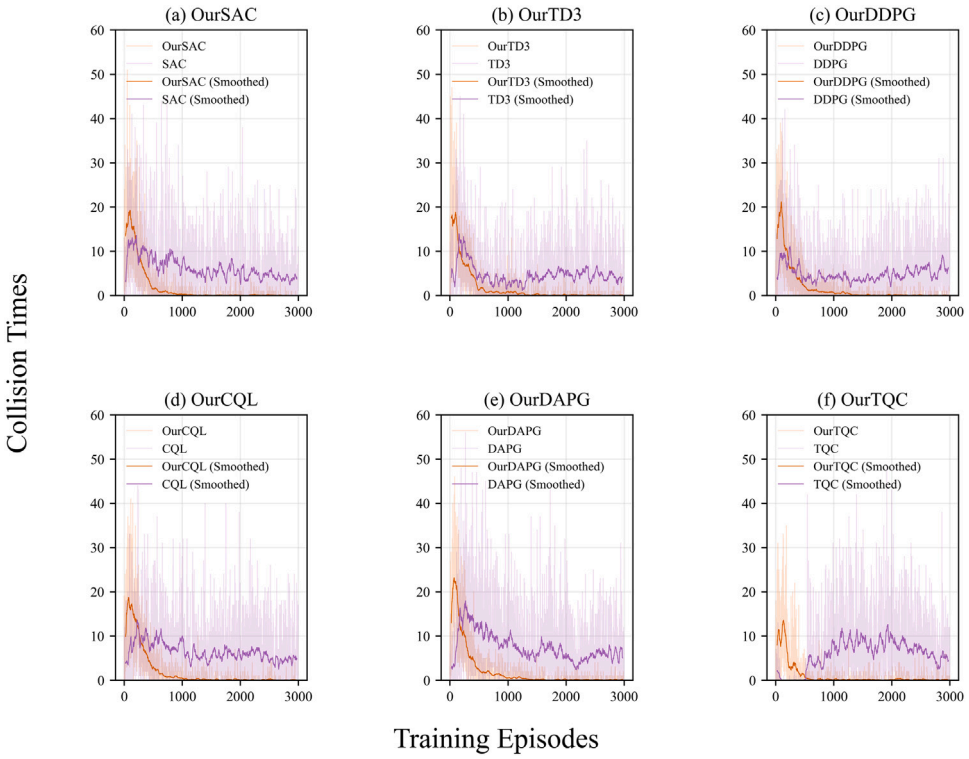


Fig. 10. Evolution curves of per-episode collision counts for potential-collision modules of robotic manipulators during adaptive motion planning, comparing the TL-ARS-TDKDQ framework with other baseline algorithms.

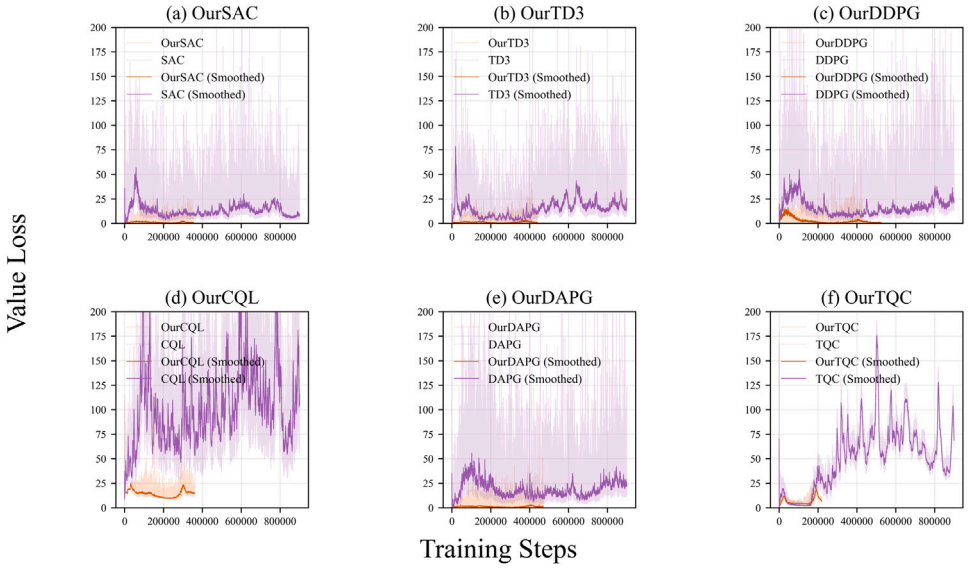


Fig. 11. Comparative analysis of value loss trends between algorithms integrated with the TL-ARS-TDKDQ framework and baseline algorithms in adaptive motion planning for robotic manipulators operating in dynamic environments.

generalization in complex dynamic environments (see Fig. 22). This comparative experiment further confirms that the adaptive adjustment mechanism effectively enhances both the generalization and robustness of the manipulator's control system. The results are presented in Table 5.

4.2.3. Ablation study on Q-value knowledge distillation based on batch experience sample TD error

To comprehensively evaluate the generality and effectiveness of the proposed TDKDQ framework in non-stationary reward environments, we integrated it into multiple continuous-action reinforcement learning setups and conducted comparative experiments against their corresponding baseline versions. The experimental environment was configured as a dynamic reward disturbance (ARS) task scenario. We employed convergence speed during training as well as the average sliding reward standard deviation σ^{R}_{avg} and the maximum sliding reward standard deviation $\sigma^{R}_{\max}$, which reflect reward volatility, as evaluation metrics to systematically analyze the performance improvements of the TDKDQ framework across different algorithms. In addition, we conducted 20 independent training runs for each configuration and

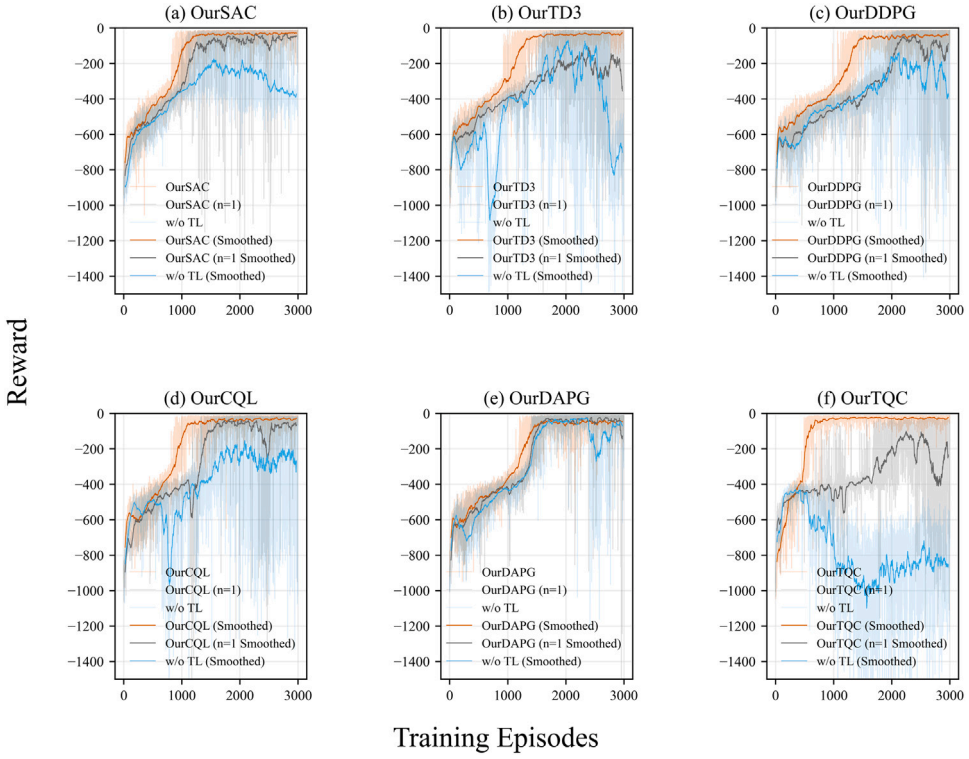


Fig. 12. Evolution of per-episode rewards for motion planners in dynamic robotic manipulation tasks under ablation settings of the Trend Learning module, including structural ablation (with/without TL) and temporal context ablation ($n = 1$).

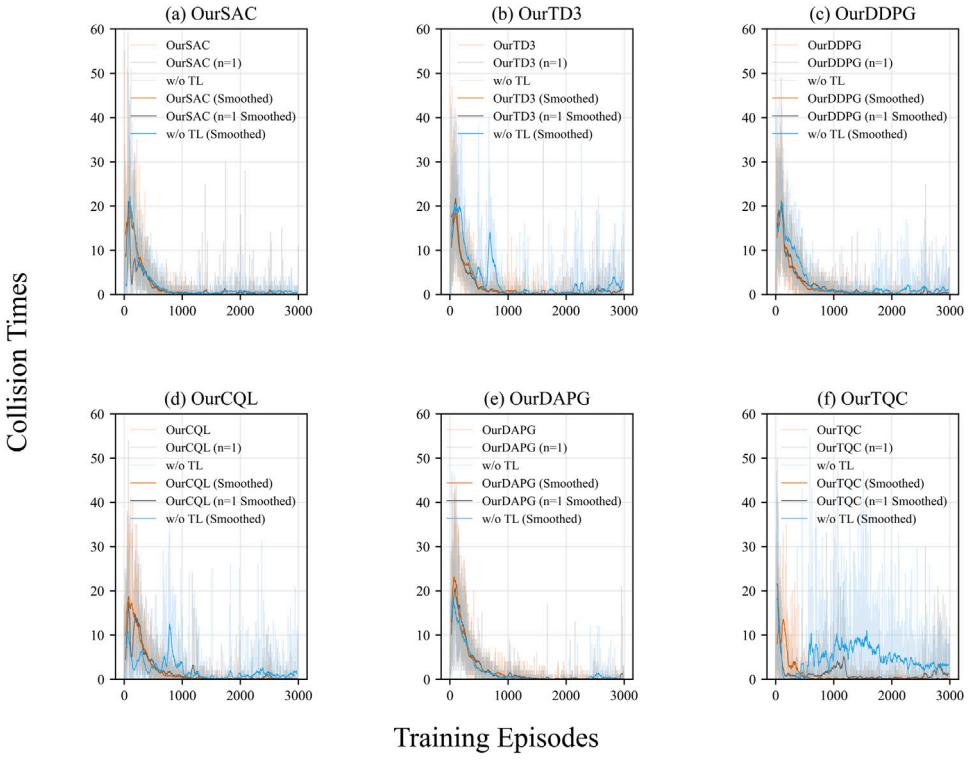


Fig. 13. Evolution of per-episode collision numbers for motion planners in dynamic robotic manipulation tasks under ablation settings of the Trend Learning module, including structural ablation (with/without TL) and temporal context ablation ($n = 1$).

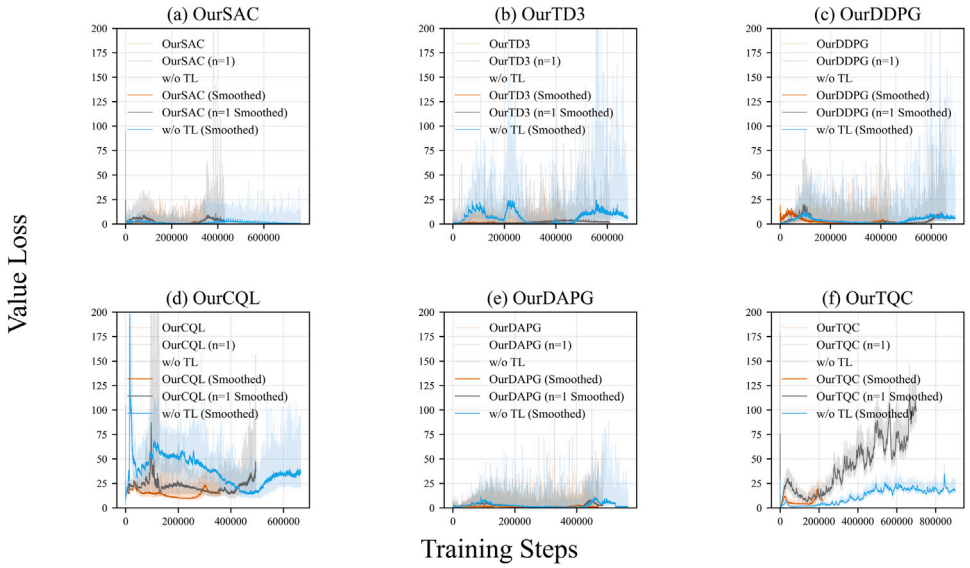


Fig. 14. Evolution of per-step value loss for motion planners in dynamic robotic manipulation tasks under ablation settings of the Trend Learning module, including structural ablation (with/without TL) and temporal context ablation ($n = 1$).

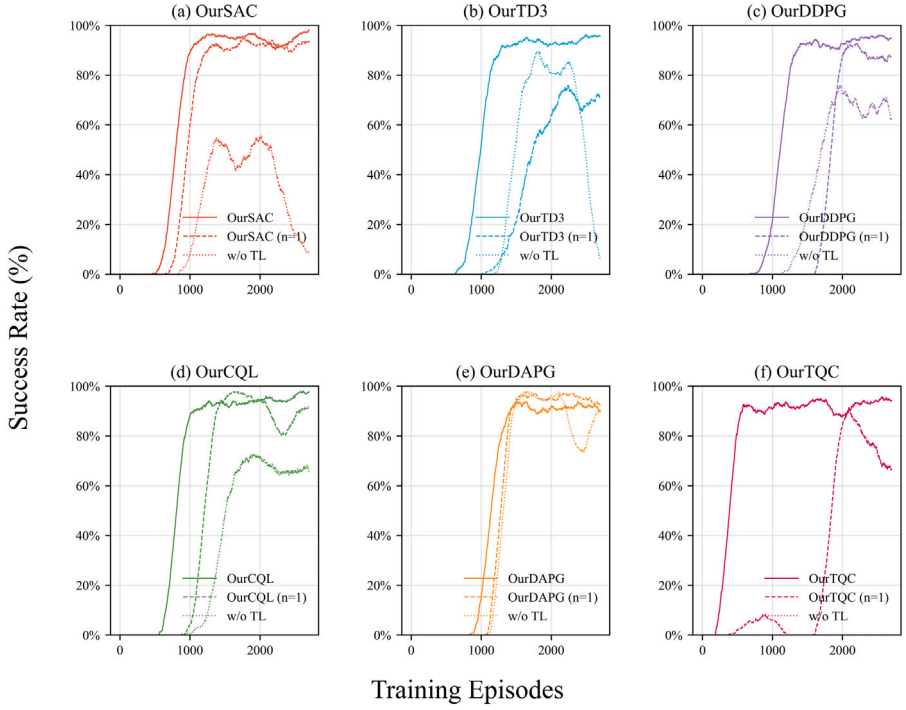


Fig. 15. Evolution of sliding-window success rates for motion planners in dynamic robotic manipulation tasks under ablation settings of the Trend Learning module, including structural ablation (with/without TL) and temporal context ablation ($n = 1$).

computed the convergence success rate (S_{rate}), defined as the proportion of runs in which the algorithm achieved stable convergence, to quantitatively assess training robustness under dynamic conditions. The results are presented in Table 6.

The experimental results demonstrate that integrating TDKDQ into various continuous-action reinforcement learning frameworks significantly improves policy stability and task completion in robotic manipulator positioning under dynamic obstacle avoidance, thereby validating the generality and effectiveness of the TD-error-guided target value distillation mechanism in complex, non-stationary robotic environments. In contrast, although the original algorithms initially benefit from higher policy exploration and relatively stable reward mechanisms in low-conflict scenarios (primarily localization control), which partially mitigates value function estimation disturbances caused by non-stationary rewards and leads to preliminary policy performance improvements, this mitigation effect progressively diminishes as training proceeds (see Figs. 23–27). Specifically, as the policies gradually converge in robotic manipulator control tasks, exploration decreases notably and policy behaviors tend to become deterministic, while the non-stationary rewards induce irreversible shifts in the state–action distribution. This causes network training in later stages to be prone to local overfitting or misleading updates, ultimately resulting in policy degradation or even collapse, thereby failing to achieve the task

Table 4
Performance comparison of different policy gradient algorithms under settings without the Trend Learning (TL) module and with varying temporal sequence lengths n in the TL-enabled case.

Algorithm	Metric	Without TL	Sequence Length n(with TL)					
			1	2	3	4	7	10
Our SAC	$\overline{R}_{500}$	−356.88	−58.11	−33.85	−30.32	−68.14	−253.87	−279.43
	$\sigma_{R_{500}}$	93.65	113.97	32.86	25.03	63.34	126.19	129.82
	S_{rate} (%)	13.9	86.7	94.2	94.5	82.1	36.5	32.5
Our TD3	$\overline{R}_{500}$	−538.26	−234.08	−33.04	−33.03	−69.00	−249.60	−342.64
	$\sigma_{R_{500}}$	343.80	251.34	26.17	26.91	69.44	131.40	101.86
	S_{rate} (%)	8.9	52.3	93.8	92.4	62.8	10.5	0.0
Our DDPG	$\overline{R}_{500}$	−270.71	−124.50	−59.31	−43.23	−74.80	−284.56	−354.28
	$\sigma_{R_{500}}$	292.08	238.01	92.33	40.14	77.50	119.32	84.60
	S_{rate} (%)	66.8	83.4	87.1	91.3	44.8	16.2	2.8
Our CQL	$\overline{R}_{500}$	−261.57	−74.35	−36.12	−31.51	−321.86	−361.44	−283.89
	$\sigma_{R_{500}}$	296.52	117.96	34.50	29.50	130.92	84.99	43.70
	S_{rate} (%)	68.4	90.0	92.9	92.7	82.1	45.9	29.0
Our DAPG	$\overline{R}_{500}$	−125.03	−47.85	−36.47	−48.61	−196.67	−243.91	−291.55
	$\sigma_{R_{500}}$	187.04	131.98	27.64	49.00	137.12	168.21	72.52
	S_{rate} (%)	84.8	89.0	92.0	91.0	82.8	19.5	4.5
Our TQC	$\overline{R}_{500}$	−826.98	−260.85	−25.78	−27.49	−164.39	319.69	389.26
	$\sigma_{R_{500}}$	220.38	283.32	17.12	21.64	111.81	187.22	80.72
	S_{rate} (%)	0.0	67.2	94.6	91.4	21.3	0.0	0.0

Table 5
Evaluation of generalization performance: Success rates (%) of different algorithms with and without adaptive reward shaping (ARS) under stricter testing conditions.

Algorithm	Steps(T)	Accuracy (ϵ)			
		0.015	0.020	0.025	0.030
Our SAC Our SAC w/o ARS	3	94.0/42.5	95.0/67.0	96.0/79.5	95.5/88.0
	5	88.5/6.5	92.5/20.5	94.5/47.5	95.5/68.5
	7	74.5/1.0	87.0/3.0	97.0/12.0	96.0/26.5
	9	58.5/0.0	82.0/0.5	92.5/3.0	95.0/8.0
Our TD3 Our TD3 w/o ARS	3	88.5/46.5	94.0/74.0	95.0/83.5	93.0/89.5
	5	75.5/7.5	84.5/25.5	93.5/46.0	90.0/63.5
	7	58.0/0.0	78.0/4.0	89.5/11.5	90.5/28.5
	9	41.0/0.0	71.0/0.0	85.5/0.5	85.5/9.5
Our DDPG Our DDPG w/o ARS	3	87.5/58.0	90.5/83.5	94.5/91.5	94.0/92.5
	5	72.5/23.0	83.0/54.0	89.0/75.0	89.5/83.5
	7	57.0/3.0	77.0/22.5	87.0/41.5	88.0/56.0
	9	31.0/0.0	62.0/0.5	83.0/11.5	82.0/26.0
Our CQL Our CQL w/o ARS	3	93.0/19.0	97.0/50.0	96.0/68.0	97.5/80.0
	5	86.5/1.5	92.0/9.0	92.5/25.5	96.5/38.5
	7	73.0/0.0	89.5/1.0	93.0/5.0	95.0/11.0
	9	53.0/0.0	84.5/0.0	91.0/2.0	95.0/4.0
Our DAPG Our DAPG w/o ARS	3	86.0/45.5	93.0/67.0	93.0/77.5	94.0/89.0
	5	72.0/8.0	85.0/25.5	91.0/38.0	92.0/55.5
	7	46.5/0.0	70.0/8.5	82.5/20.0	86.5/19.0
	9	23.5/0.5	57.0/2.0	71.5/4.5	76.0/7.0
Our TQC Our TQC w/o ARS	3	86.0/11.0	91.0/26.5	96.0/48.5	97.5/71.0
	5	71.0/0.0	86.5/5.0	92.0/15.5	92.0/23.5
	7	53.5/0.0	71.5/1.0	78.0/6.0	89.5/1.0
	9	29.0/0.0	52.5/0.0	63.5/0.0	75.5/3.0

objectives. This phenomenon is particularly pronounced in traditional target networks lacking mechanisms for historical knowledge buffering and estimation stability, indicating the difficulty of conventional target network designs in addressing long-term disturbance accumulation arising in dynamic reward environments typically encountered in robotic manipulator operations.

The proposed TDKDQ module essentially functions as a TD-error-driven target value distillation mechanism tailored for robotic manipulator control, achieving a dynamic balance between inheriting historical knowledge and rapidly adapting to environmental changes through multi-source Q-value fusion. It introduces a structurally consistent teacher target network, periodically cloning parameters from the main critic network to establish an auxiliary supervision path with long-term memory capacity, thereby preserving stable value estimations from early training stages. The core mechanism employs the TD error as a measure of policy uncertainty to adaptively regulate the fusion weights between the teacher target Q-values and the traditional target network outputs. When the TD error is large, the model relies more on the teacher network to suppress value function oscillations caused by drastic policy adjustments in robotic positioning and obstacle avoidance control. Conversely, when the error is small, it increases confidence in the current estimates, improving sensitivity to real-time environmental feedback. To further support the effectiveness of this mechanism, we present convergence plots illustrating the differences in Q-values among the target critic, teacher critic, and soft Q during training, providing an intuitive demonstration of the stabilization and convergence behavior of the value function (see Fig. 28). This soft-label construction strategy effectively bridges historical knowledge constraints with current estimate guidance, forming a robust value

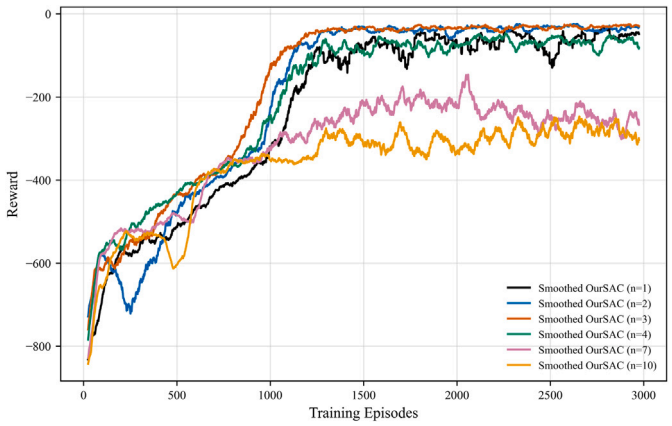


Fig. 16. Evolution of per-episode rewards under the Trend Learning mechanism with different temporal sequence lengths in adaptive robotic motion planning tasks.

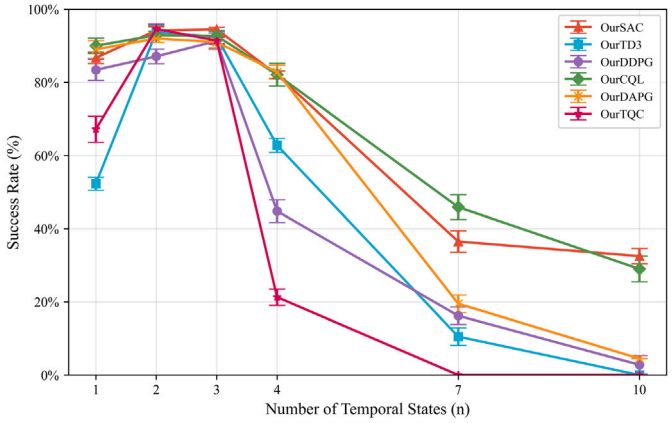


Fig. 17. Evolution of training success rates under the Trend Learning mechanism with varying temporal sequence lengths in adaptive robotic motion planning tasks.

Table 6
Stability evaluation results of TDKDQ module integrated into different policy gradient algorithms.

Algorithm	N_{conv}	σ^R_{avg}	σ^R_{max}	P_{rate} (%)
Our SAC	1373	45.08	117.49	100.0
Our SAC w/o TDKDQ	—	93.13	228.19	15.0
Our TD3	1419	44.29	115.29	100.0
Our TD3 w/o TDKDQ	—	87.44	156.36	20.0
Our DDPG	1777	56.22	121.18	90.0
Our DDPG w/o TDKDQ	—	141.29	321.88	15.0
Our CQL	1184	54.58	124.97	100.0
Our CQL w/o TDKDQ	—	135.20	359.05	25.0
Our DAPG	1858	62.13	118.36	95.0
Our DAPG w/o TDKDQ	—	94.47	174.63	20.0
Our TQC	1033	42.72	157.67	100.0
Our TQC w/o TDKDQ	—	50.93	146.17	55.0

Note: N_{conv} indicates the number of episodes required for convergence. The symbol — denotes cases where the agent failed to converge within the predefined training episodes.

function supervision mechanism well-suited for non-stationary, multi-objective robotic manipulator training environments characterized by dynamic task variations and non-stationary environment transition dynamics.

5. Conclusion

In this paper, we proposed a novel TL-ARS-TDKDQ framework for robotic dynamic obstacle avoidance and target localization under non-stationary environments. The framework integrates Trend Learning (TL) and a TD-error-based Q-Value Knowledge Distillation (TD-KDQ) mechanism within an Adaptive Reward Shaping (ARS) scheme. It systematically tackles critical issues in robotic manipulator multi-task control for dynamic obstacle avoidance and positioning accuracy in highly dynamic and partially observable environments, including decision bias under partially observable Markov decision processes (POMDPs), conflicts among multiple task objectives, and instability in policy optimization due to non-stationary environmental conditions.

To validate the effectiveness of the proposed framework, we integrated it into six mainstream continuous control reinforcement learning algorithms and conducted comprehensive comparative experiments using a 7-DOF KUKA robotic manipulator within the CoppeliaSim simulation platform. Experimental results demonstrate that, compared with the original algorithms, the TL-ARS-TDKDQ-integrated methods achieve significant improvements in policy convergence speed, training stability, and overall task performance. These findings verify the generality and effectiveness of the proposed framework for dynamic obstacle avoidance and localization control in robotic systems under dynamic multi-task environments.

In future work, we will advance our framework toward real-world robotic deployment by integrating visual perception modules that fuse image data with proprioceptive state information, thereby enhancing environmental awareness and robustness under real sensory noise and dynamic uncertainty. Moreover, we plan to extend the proposed approach to human–robot collaborative scenarios in unstructured and dynamic environments, aiming to achieve safe, efficient, and adaptive motion planning in practical applications.

CRediT authorship contribution statement

Dexian Wang: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Peng Zhang:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Methodology, Funding acquisition, Data curation, Conceptualization. **Pengfei Ding:** Writing – review & editing, Validation, Formal analysis, Data curation. **Junliang Wang:** Validation, Funding acquisition. **Jie Zhang:** Resources, Project administration, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We thank the editors and the anonymous reviewers for their fruitful comments and suggestions in improving the quality of this paper. This work was supported by the National Natural Science Foundation of China [grant number 52005099]; Shanghai Soft Science Research Funds [grant number 25692107300]; the Fundamental Research Funds for the Central Universities, China [grant number 2232025G-14].

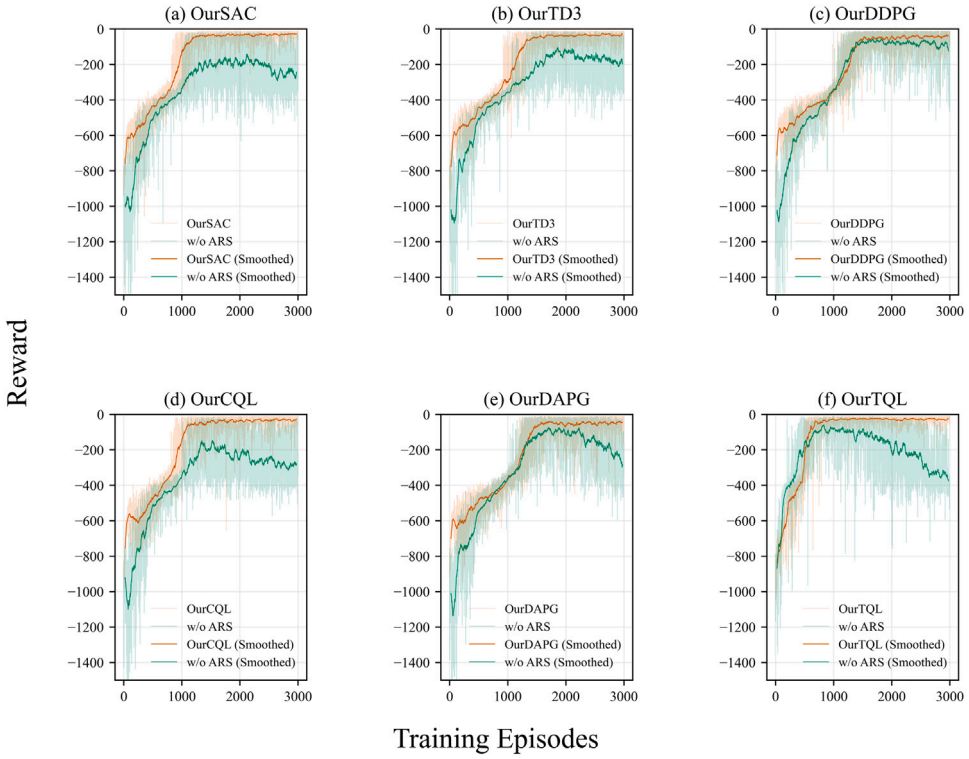


Fig. 18. Evolution of per-episode rewards with and without the Adaptive Reward Shaping module in adaptive robotic motion planning tasks under dynamic environments.

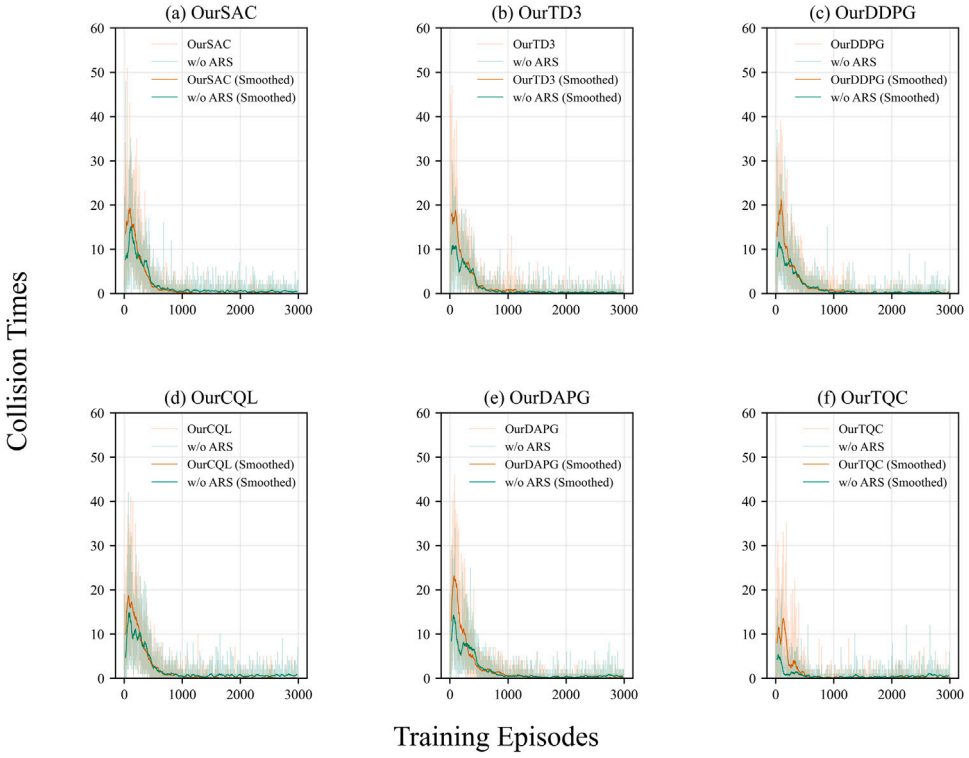


Fig. 19. Evolution of per-episode collision numbers with and without the Adaptive Reward Shaping module in adaptive robotic motion planning tasks under dynamic environments.

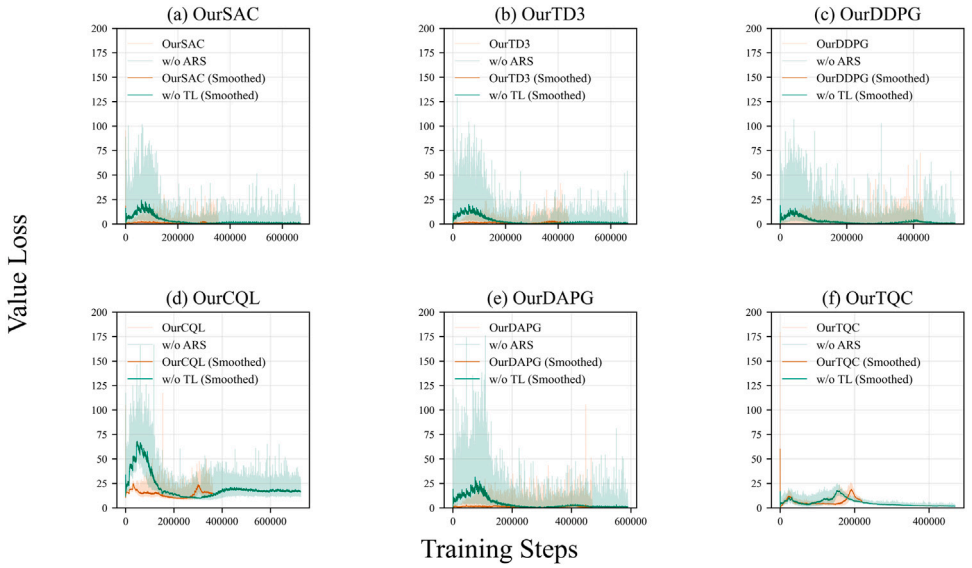


Fig. 20. Evolution of per-step value loss with and without the Adaptive Reward Shaping module in adaptive robotic motion planning tasks under dynamic environments.

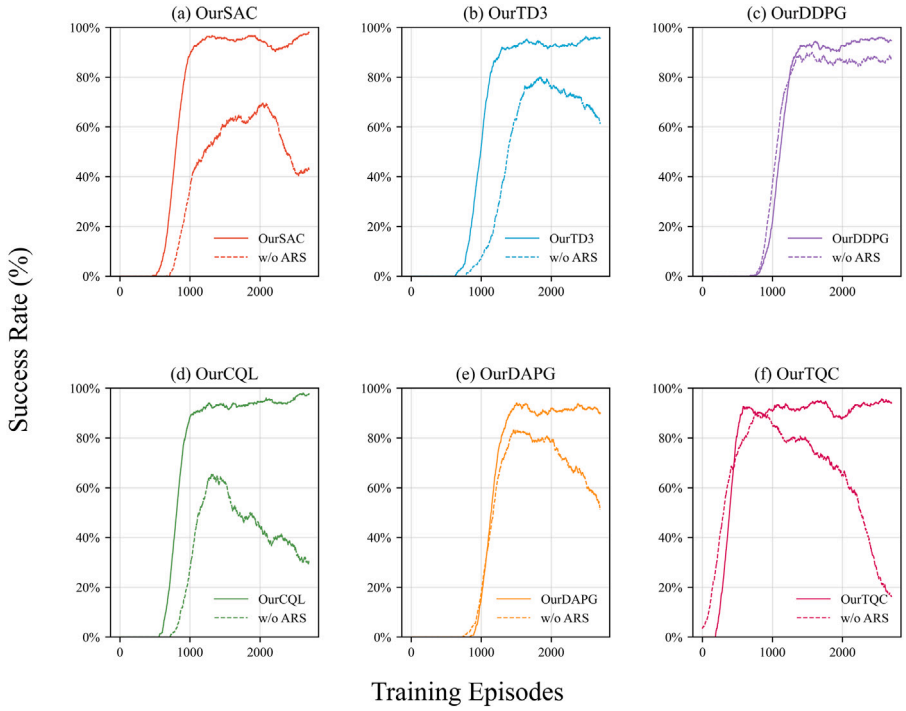


Fig. 21. Evolution of training success rates with and without the Adaptive Reward Shaping module in adaptive robotic motion planning tasks under dynamic environments.

Appendix A. Convergence and stability analysis of TD-guided soft target Q under ARS

1. Problem setup

Consider a finite state–action space:

$$|S| < \infty, \quad |A| < \infty.$$

A fixed policy π defines the action-value function $Q_\pi(s,a)$. The target Critic and teacher Critic are denoted by Q_t^θ and Q_t^ϕ, respectively.

The teacher Critic is a delayed copy of the target Critic:

$$Q_t^\phi = Q_{t-k}^\theta, \quad k > 0.$$

The target Critic is updated with soft updates:

$$Q_{t+1}^\theta = (1-\tau)Q_t^\theta + \tau T_\pi Q_t^\theta, \quad 0 < \tau < 1.$$

The Bellman operator is defined as:

$$(T_\pi Q)(s,a) = \mathbb{E}_{s' \sim P, a' \sim \pi}[r_t + \gamma Q(s',a')], \quad 0 < \gamma < 1.$$

The ARS-induced non-stationary reward is:

$$r_t = r_{\text{error}} + \sum_{i=1}^{6} r_{\text{collision},i} + r_{\text{keep}}, \quad |\Delta r_t| \le \epsilon.$$

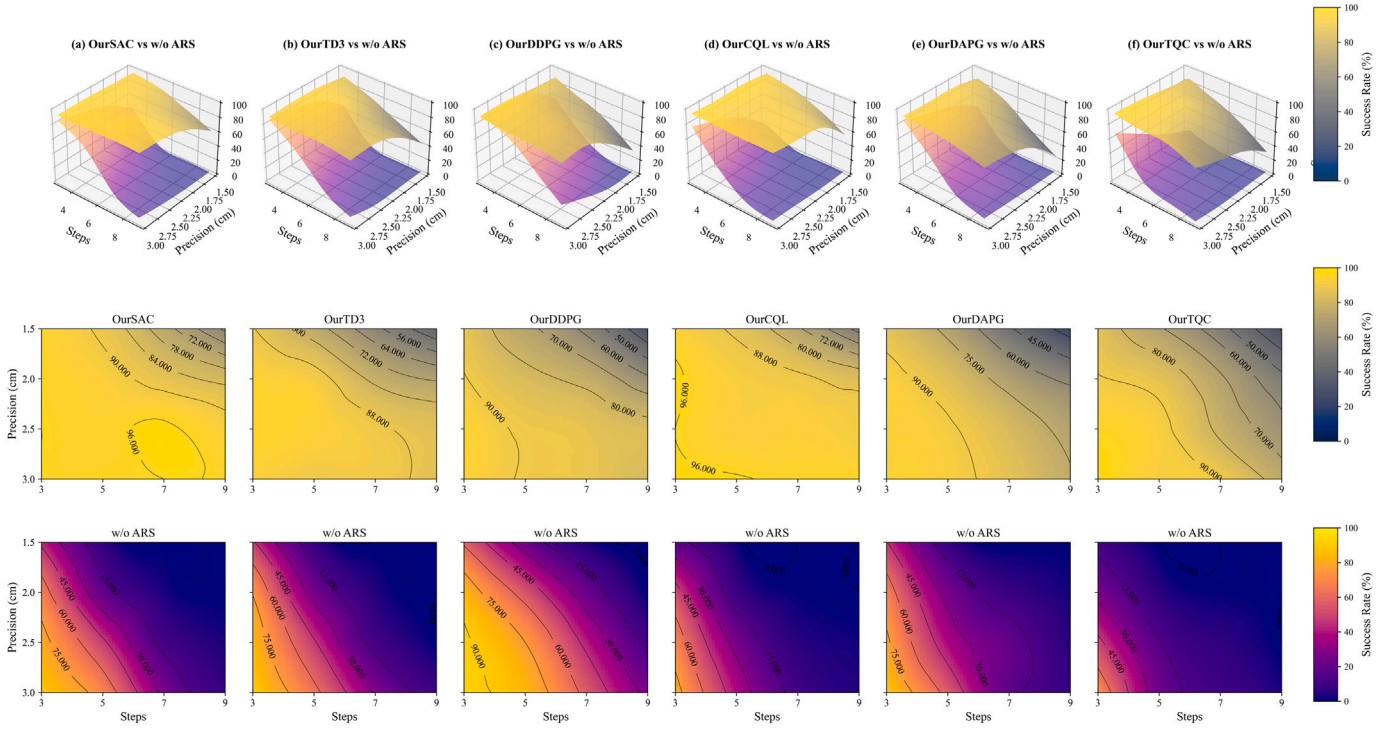


Fig. 22. Success rate comparison with and without the Adaptive Reward Shaping (ARS) module in task generalization tests for adaptive robotic arm motion planning under dynamic environments. Subfigures show 3D success-rate surfaces and the corresponding 2D heatmaps for both variants across different precision levels and step settings.

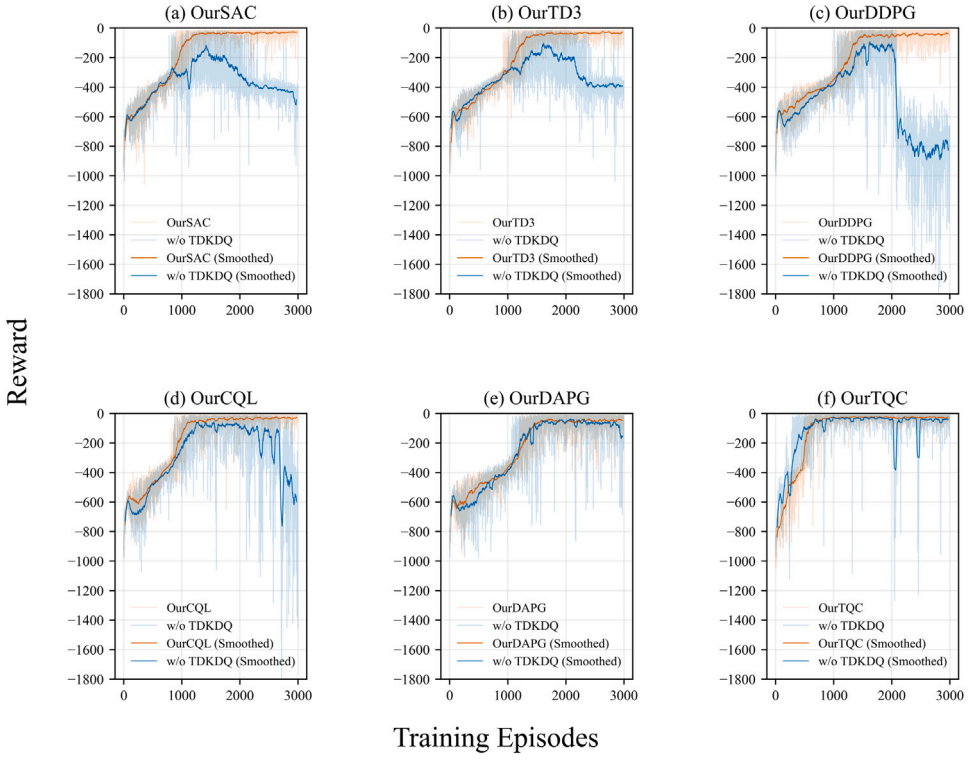


Fig. 23. Evolution of per-episode rewards with and without the TD Error-based Q-Value Distillation module in adaptive robotic motion planning tasks under dynamic environments.

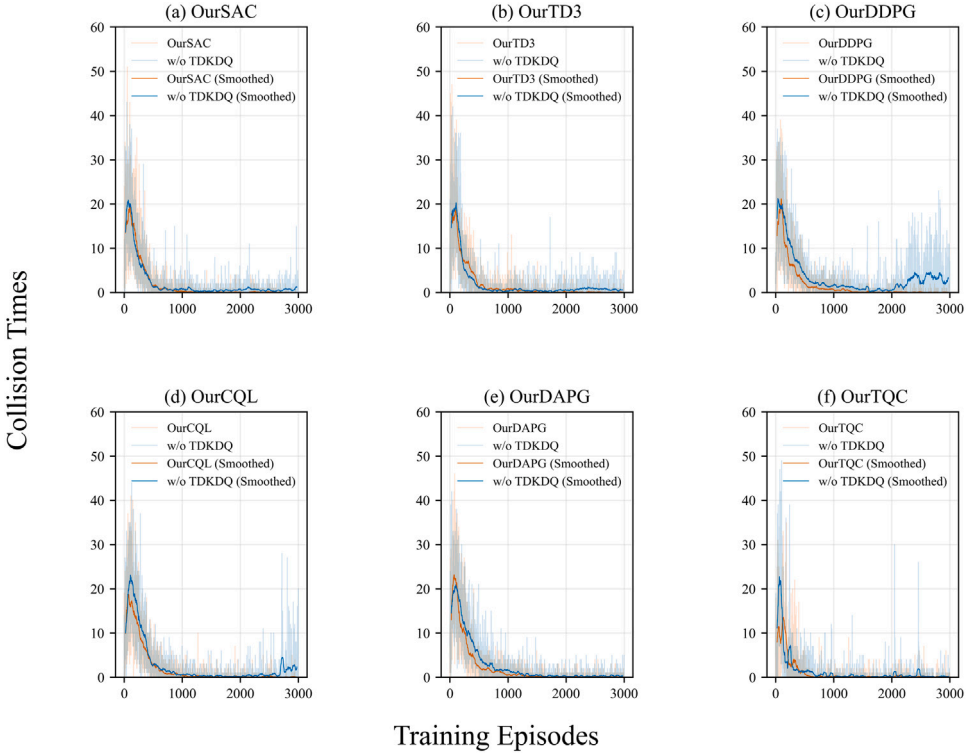


Fig. 24. Evolution of per-episode collision numbers with and without the TD Error-based Q-Value Distillation module in adaptive robotic motion planning tasks under dynamic environments.

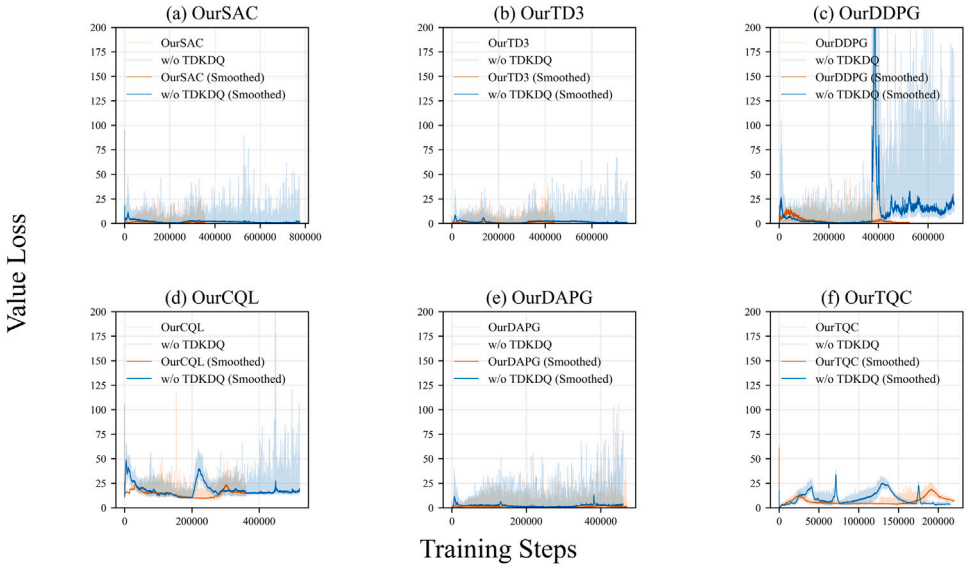


Fig. 25. Evolution of per-step value loss with and without the TD Error-based Q-Value Distillation module in adaptive robotic motion planning tasks under dynamic environments.

The TD-guided soft target Q-value is defined as:

$$Q_t^{\text{soft}} = \beta_t Q_t^{\phi} + (1-\beta_t) Q_t^{\theta}, \quad \beta_t = f(\text{TD error norm}).$$

2. Convergence analysis

2.1 convergence of the target critic

Define the error:

$$\Delta_t^{\theta} := Q_t^{\theta} - Q_{\pi}.$$

The iterative relationship is:

$$\Delta_{t+1}^{\theta} = (1-\tau)\Delta_t^{\theta} + \tau(T_{\pi} Q_t^{\theta} - T_{\pi} Q_{\pi}).$$

Using the contraction property of the Bellman operator:

$$\|T_{\pi} Q_1 - T_{\pi} Q_2\|_{\infty} \le \gamma \|Q_1 - Q_2\|_{\infty},$$

we obtain:

$$\|\Delta_{t+1}^{\theta}\|_{\infty} \le (1 - \tau(1-\gamma))\|\Delta_t^{\theta}\|_{\infty}.$$

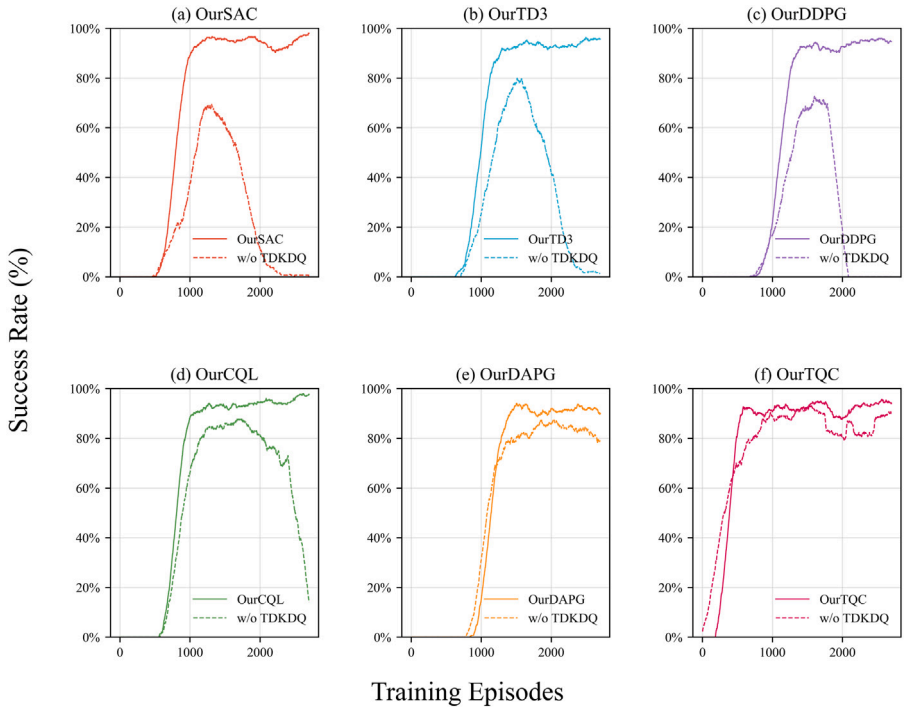


Fig. 26. Evolution of training success rates with and without the TD Error-based Q-Value Distillation module in adaptive robotic motion planning tasks under dynamic environments.

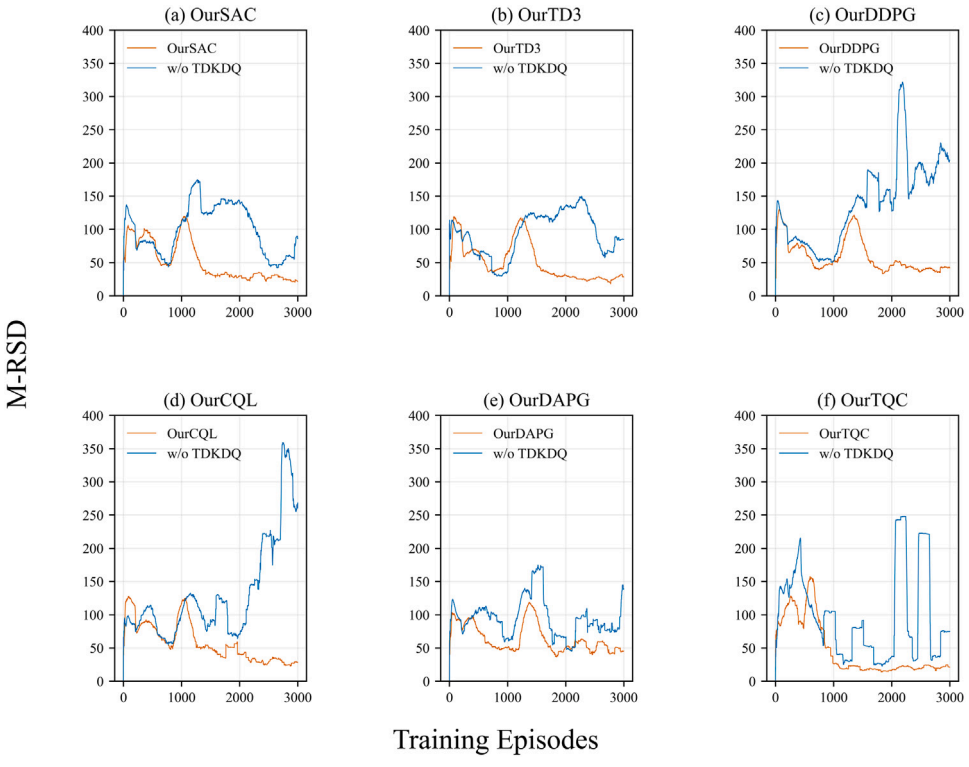


Fig. 27. Evolution of sliding-window reward standard deviations with and without the TD Error-based Q-Value Distillation module in adaptive robotic motion planning tasks under dynamic environments.

Hence:

$$\lim_{t\to\infty} \Delta_t^{\theta} = 0 \quad \Longrightarrow \quad \lim_{t\to\infty} Q_t^{\theta} = Q_{\pi}.$$

2.2 convergence of the teacher critic

The teacher Critic is updated with a delay of k steps:

$$Q_t^{\phi} = Q_{t-k}^{\theta}.$$

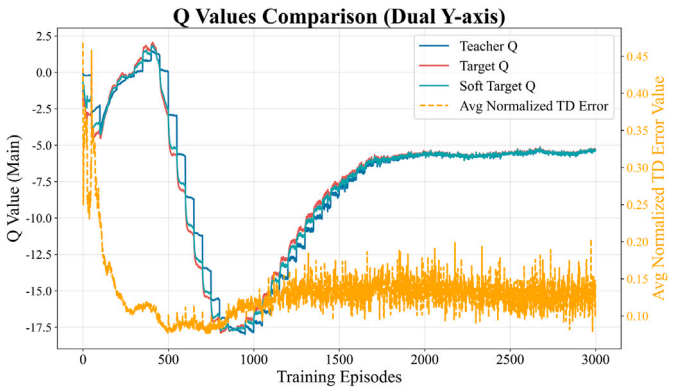


Fig. 28. Visualization of the adaptive Q-value fusion among the teacher target Q, target Q, and soft Q guided by batch-normalized TD errors.

Using the convergence of the target Critic:

$$\lim_{t\to\infty} Q_t^{\phi} = \lim_{t\to\infty} Q_{t-k}^{\theta} = Q_{\pi}.$$

2.3 convergence of the soft target critic

The soft target error is:

$$\Delta_t^{\text{soft}} := Q_t^{\text{soft}} - Q_{\pi} = \beta_t \Delta_t^{\phi} + (1-\beta_t)\Delta_t^{\theta}.$$

Taking the infinity norm:

$$\|\Delta_t^{\text{soft}}\|_{\infty} \le \beta_t \|\Delta_t^{\phi}\|_{\infty} + (1-\beta_t)\|\Delta_t^{\theta}\|_{\infty}.$$

From Sections 2.1 and 2.2:

$$\lim_{t\to\infty} \|\Delta_t^{\theta}\|_{\infty} = \lim_{t\to\infty} \|\Delta_t^{\phi}\|_{\infty} = 0,$$

therefore:

$$\lim_{t\to\infty} \|\Delta_t^{\text{soft}}\|_{\infty} = 0 \quad \Longrightarrow \quad \lim_{t\to\infty} Q_t^{\text{soft}} = Q_{\pi}.$$

3. Stability analysis (teacher network)

Define the variances:

$$\sigma_{\theta}^2 := \text{Var}[Q_t^{\theta}], \quad \sigma_{\phi}^2 := \text{Var}[Q_t^{\phi}] < \sigma_{\theta}^2.$$

The variance of the soft target Q-value:

$$\text{Var}[Q_t^{\text{soft}}] = \beta_t^2\sigma_{\phi}^2 + (1-\beta_t)^2\sigma_{\theta}^2 + 2\beta_t(1-\beta_t)\text{Cov}(Q_t^{\phi}, Q_t^{\theta}),$$

with covariance upper bound:

$$|\text{Cov}(Q_t^{\phi}, Q_t^{\theta})| \le \sigma_{\phi}\sigma_{\theta}.$$

Thus:

$$\text{Var}[Q_t^{\text{soft}}] \le \max(\sigma_{\phi}^2, \sigma_{\theta}^2),$$

showing that the teacher Critic smooths historical information and reduces fluctuations from non-stationary ARS rewards.

Data availability

Data will be made available on request.

References

Agha-mohammadi, A.-a., Agarwal, S., Kim, S.-K., Chakravorty, S., Amato, N.M., 2018. SLAP: Simultaneous localization and planning under uncertainty via dynamic replanning in belief space. IEEE Trans. Robot. 34 (5), 1195–1214.

Alegre, L.N., Serifi, A., Grandia, R., Müller, D., Knoop, E., Bächer, M., 2025. AMOR: Adaptive character control through multi-objective reinforcement learning. arXiv preprint arXiv:2505.23708.

Bai, H., Cheng, R., Jin, Y., 2023. Evolutionary reinforcement learning: A survey. Intell. Comput. 2, 0025.

Berducci, L., Aguilar, E.A., Ničković, D., Grosu, R., 2021. Hierarchical potential-based reward shaping from task specifications. arXiv preprint arXiv:2110.02792.

Bing, Z., Lerch, D., Huang, K., Knoll, A., 2022. Meta-reinforcement learning in non-stationary and dynamic environments. IEEE Trans. Pattern Anal. Mach. Intell. 45 (3), 3476–3491.

Blackmore, L., Ono, M., Bektassov, A., Williams, B.C., 2010. A probabilistic particle-control approximation of chance-constrained stochastic predictive control. IEEE Trans. Robot. 26 (3), 502–517.

Buciluǎ, C., Caruana, R., Niculescu-Mizil, A., 2006. Model compression. In: Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 535–541.

Chai, J., Chen, E., Fan, J., 2025. Deep transfer Q-learning for offline non-stationary reinforcement learning. arXiv preprint arXiv:2501.04870.

Chen, G., Liu, D., Wang, Y., Jia, Q., Zhang, X., 2018. Path planning method with obstacle avoidance for manipulators in dynamic environment. Int. J. Adv. Robot. Syst. 15 (6), 1729881418820223.

Chen, P., Pei, J., Lu, W., Li, M., 2022. A deep reinforcement learning based method for real-time path planning and dynamic obstacle avoidance. Neurocomputing 497, 64–75.

Cheung, W.C., Simchi-Levi, D., Zhu, R., 2020. Reinforcement learning for non-stationary markov decision processes: The blessing of (more) optimism. In: International Conference on Machine Learning. PMLR, pp. 1843–1854.

Cui, J., Liu, T., Meng, Z., Yu, J., Song, R., Zhang, W., Zhu, Y., Huang, S., 2025. Grove: A generalized reward for learning open-vocabulary physical skill. In: Proceedings of the Computer Vision and Pattern Recognition Conference. pp. 15781–15790.

Dai, S., Schaffert, S., Jasour, A., Hofmann, A., Williams, B., 2019. Chance constrained motion planning for high-dimensional robots. In: 2019 International Conference on Robotics and Automation. ICRA, IEEE, pp. 8805–8811.

Dewangan, P., Phaniteja, S., Krishna, K.M., Sarkar, A., Ravindran, B., 2018. Digrad: Multi-task reinforcement learning with shared actions. arXiv preprint arXiv:1802.10463.

Ding, Z.-J., Meng, X.-J., Zhang, L.-X., Guo, Y.-B., Chen, M.-Q., Zhen, S.-H., 2025. Variable Sampling Area RRT algorithm based on limiting joint angle for robotic arm path planning. IEEE Access.

Duan, Y., Schulman, J., Chen, X., Bartlett, P.L., Sutskever, I., Abbeel, P., 2016. RI^2: Fast reinforcement learning via slow reinforcement learning. arXiv preprint arXiv:1611.02779.

Ekrem, Ö., Aksoy, B., 2023. Trajectory planning for a 6-axis robotic arm with particle swarm optimization algorithm. Eng. Appl. Artif. Intell. 122, 106099.

Escontrela, A., Peng, X.B., Yu, W., Zhang, T., Iscen, A., Goldberg, K., Abbeel, P., 2022. Adversarial motion priors make good substitutes for complex reward functions. In: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, IEEE, pp. 25–32.

Finn, C., Abbeel, P., Levine, S., 2017. Model-agnostic meta-learning for fast adaptation of deep networks. In: International Conference on Machine Learning. PMLR, pp. 1126–1135.

Fujimoto, S., Hoof, H., Meger, D., 2018. Addressing function approximation error in actor-critic methods. In: International Conference on Machine Learning. PMLR, pp. 1587–1596.

Haarnoja, T., Zhou, A., Abbeel, P., Levine, S., 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: International Conference on Machine Learning. Pmlr, pp. 1861–1870.

Han, R., Chen, S., Wang, S., Zhang, Z., Gao, R., Hao, Q., Pan, J., 2022. Reinforcement learned distributed multi-robot navigation with reciprocal velocity obstacle shaped rewards. IEEE Robot. Autom. Lett. 7 (3), 5896–5903.

Hausknecht, M.J., Stone, P., 2015. Deep recurrent Q-learning for partially observable mdps.. In: AAAI Fall Symposia, vol. 45, p. 141.

Hinton, G., Vinyals, O., Dean, J., 2015. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531.

Hoerger, M., Kurniawati, H., Elfes, A., 2023. Multilevel Monte Carlo for solving POMDPs on-line. Int. J. Robot. Res. 42 (4–5), 196–213.

Hou, L., Pang, L., Hong, X., Lan, Y., Ma, Z., Yin, D., 2020. Robust reinforcement learning with wasserstein constraint. arXiv preprint arXiv:2006.00945.

Huang, B., Jin, Y., 2022. Reward shaping in multiagent reinforcement learning for self-organizing systems in assembly tasks. Adv. Eng. Inform. 54, 101800.

Kapturowski, S., Ostrovski, G., Quan, J., Munos, R., Dabney, W., 2018. Recurrent experience replay in distributed reinforcement learning. In: International Conference on Learning Representations.

Kiemel, J., Righetti, L., Kröger, T., Asfour, T., 2024. Safe reinforcement learning of robot trajectories in the presence of moving obstacles. IEEE Robot. Autom. Lett..

Kim, D., Hong, M., Park, J., Oh, S., 2024. Conflict-averse gradient aggregation for constrained multi-objective reinforcement learning. arXiv preprint arXiv:2403.00282.

Kochenderfer, M.J., 2015. Decision Making Under Uncertainty: Theory and Application. MIT Press.

Kong, L., He, W., Yang, W., Li, Q., Kaynak, O., 2020. Fuzzy approximation-based finite-time control for a robot with actuator saturation under time-varying constraints of work space. IEEE Trans. Cybern. 51 (10), 4873–4884.

Kuffner, J.J., LaValle, S.M., 2000. RRT-connect: An efficient approach to single-query path planning. In: Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No. 00CH37065), vol. 2, IEEE, pp. 995–1001.

Lauri, M., Hsu, D., Pajarinen, J., 2022. Partially observable markov decision processes in robotics: A survey. IEEE Trans. Robot. 39 (1), 21–40.

Li, X., Liu, H., Dong, M., 2021. A general framework of motion planning for redundant robot manipulator based on deep reinforcement learning. IEEE Trans. Ind. Inform. 18 (8), 5253–5263.

Li, X., Tang, B., Li, H., 2024. Adaer: An adaptive experience replay approach for continual lifelong learning. Neurocomputing 572, 127204.

Lim, J., Ha, S., Choi, J., 2020. Prediction of reward functions for deep reinforcement learning via Gaussian process regression. IEEE/ASME Trans. Mechatronics 25 (4), 1739–1746.

Lim, J., Tsiotras, P., 2021. A generalized a* algorithm for finding globally optimal paths in weighted colored graphs. In: 2021 IEEE International Conference on Robotics and Automation. ICRA, IEEE, pp. 7503–7509.

Lin, H.-I., Shodiq, M.A.F., Hsieh, M.F., 2025. Robot path planning based on three-dimensional artificial potential field. Eng. Appl. Artif. Intell. 144, 110127.

Liu, S., Liu, Y., Hu, L., Zhou, Z., Xie, Y., Zhao, Z., Li, W., Gan, Z., 2024. DiffSkill: Improving reinforcement learning through diffusion-based skill denoiser for robotic manipulation. Knowl.-Based Syst. 300, 112190.

Nagabandi, A., Clavera, I., Liu, S., Fearing, R.S., Abbeel, P., Levine, S., Finn, C., 2018a. Learning to adapt in dynamic, real-world environments through meta-reinforcement learning. arXiv preprint arXiv:1803.11347.

Nagabandi, A., Finn, C., Levine, S., 2018b. Deep online learning via meta-learning: Continual adaptation for model-based RL. arXiv preprint arXiv:1812.07671.

Ortega-Aranda, D., Jimenez-Vielma, J.F., Saha, B.N., Lopez-Juarez, I., 2021. Dual-arm peg-in-hole assembly using DNN with double force/torque sensor. Appl. Sci. 11 (15), 6970.

Othman, U., Yang, E., 2023. Human–robot collaborations in smart manufacturing environments: review and outlook. Sensors 23 (12), 5663.

Pineau, J., Gordon, G., Thrun, S., et al., 2003. Point-based value iteration: An anytime algorithm for POMDPs. In: Ijcai, vol. 3, pp. 1025–1032.

Rajeswaran, A., Kumar, V., Gupta, A., Vezzani, G., Schulman, J., Todorov, E., Levine, S., 2017. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. arXiv preprint arXiv:1709.10087.

Rao, Y., Parimi, A.M., Rahul, D., Patel, D., Mythreya, Y.N., 2017. Robotics in dental implantation. Mater. Today: Proc. 4 (8), 9327–9332.

Rusu, A.A., Colmenarejo, S.G., Gulcehre, C., Desjardins, G., Kirkpatrick, J., Pascanu, R., Mnih, V., Kavukcuoglu, K., Hadsell, R., 2015. Policy distillation. arXiv preprint arXiv:1511.06295.

Saleem, M.S., Veerapaneni, R., Likhachev, M., 2024. A POMDP-based hierarchical planning framework for manipulation under pose uncertainty. arXiv preprint arXiv:2409.18775.

Sangiovanni, B., Rendiniello, A., Incremona, G.P., Ferrara, A., Piastra, M., 2018. Deep reinforcement learning for collision avoidance of robotic manipulators. In: 2018 European Control Conference. ECC, IEEE, pp. 2063–2068.

Seddaoui, A., Saaj, C.M., 2021. Collision-free optimal trajectory generation for a space robot using genetic algorithm. Acta Astronaut. 179, 311–321.

Shaker, N., 2016. Intrinsically motivated reinforcement learning: A promising framework for procedural content generation. In: 2016 IEEE Conference on Computational Intelligence and Games. CIG, IEEE, pp. 1–8.

Sheng, S., Yu, P., Parker, D., Kwiatkowska, M., Feng, L., 2024. Safe pomdp online planning among dynamic agents via adaptive conformal prediction. IEEE Robot. Autom. Lett..

Silver, D., Veness, J., 2010. Monte-Carlo planning in large POMDPs. Adv. Neural Inf. Process. Syst. 23.

Somani, A., Ye, N., Hsu, D., Lee, W.S., 2013. DESPOT: Online POMDP planning with regularization. Adv. Neural Inf. Process. Syst. 26.

Szczepanski, R., Erwinski, K., Tejer, M., Bereit, A., Tarczewski, T., 2022. Optimal scheduling for palletizing task using robotic arm and artificial bee colony algorithm. Eng. Appl. Artif. Intell. 113, 104976.

Tao, L., Zhang, J., Zhang, X., 2022. Multi-phase multi-objective dexterous manipulation with adaptive hierarchical curriculum. J. Intell. Robot. Syst. 106 (1), 1.

Thrun, S., 2002. Particle filters in robotics.. In: UAI. 2, Citeseer, pp. 511–518.

Touati, A., Vincent, P., 2020. Efficient learning in non-stationary linear markov decision processes. arXiv preprint arXiv:2010.12870.

Wang, T., Ruan, Z., Wang, Y., Chen, C., 2025. Control strategy of robotic manipulator based on multi-task reinforcement learning. Complex Intell. Syst. 11 (3), 175.

Wu, Z., Lian, W., Unhelkar, V., Tomizuka, M., Schaal, S., 2021. Learning dense rewards for contact-rich manipulation tasks. In: 2021 IEEE International Conference on Robotics and Automation. ICRA, IEEE, pp. 6214–6221.

Xie, Z., Xin, P., Dames, P., 2021. Towards safe navigation through crowded dynamic environments. In: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, IEEE, pp. 4934–4940.

Xu, J., Tian, Y., Ma, P., Rus, D., Sueda, S., Matusik, W., 2020. Prediction-guided multi-objective reinforcement learning for continuous robot control. In: International Conference on Machine Learning. PMLR, pp. 10607–10616.

Zagoruyko, S., Komodakis, N., 2016. Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer. arXiv preprint arXiv:1612.03928.

Zeng, J., Ju, R., Qin, L., Hu, Y., Yin, Q., Hu, C., 2019. Navigation in unknown dynamic environments based on deep reinforcement learning. Sensors 19 (18), 3837.

Zeng, A., Song, S., Welker, S., Lee, J., Rodriguez, A., Funkhouser, T., 2018. Learning synergies between pushing and grasping with self-supervised deep reinforcement learning. In: 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, IEEE, pp. 4238–4245.

Zhang, T., Wang, X., Liang, B., Yuan, B., 2022. Catastrophic interference in reinforcement learning: A solution based on context division and knowledge distillation. IEEE Trans. Neural Netw. Learn. Syst. 34 (12), 9925–9939.

Zhou, Z., Ren, J., Zeng, Z., Xiao, J., Zhang, X., Guo, X., Zhou, Z., Lu, H., 2023. A safe reinforcement learning approach for autonomous navigation of mobile robots in dynamic environments. CAAI Trans. Intell. Technol..

Zhou, Z., Zhu, P., Zeng, Z., Xiao, J., Lu, H., Zhou, Z., 2022. Robot navigation in a crowd by integrating deep reinforcement learning and online planning. Appl. Intell. 52 (13), 15600–15616.